SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-16607-111443

Aplikácia na organizovanie stretnutí na základe polohy používateľa

Diplomová práca

2026 **BC. Marcel Soták**

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Evidenčné číslo: FEI-16607-111443

Aplikácia na organizovanie stretnutí na základe polohy používateľa

Diplomová práca

Študijný program:	Aplikovaná informatika
Študijný odbor:	Informatika
Školiace pracovisko:	Ústav informatiky a matematiky (FEI)
Školiteľ:	Ing. Maroš Čavojský, PhD.

2026 **BC. Marcel Soták**

Slovenská technická univerzita v Bratislave
Ústav jadrového a fyzikálneho inžinierstva

Fakulta elektrotechniky a informatiky
Akademický rok: 2024/2025
Evidenčné číslo: FEI-XXXXX-YYYYYY

ZADANIE BAKALÁRSKEJ PRÁCE

Študent:	**Meno Priezvisko**
ID študenta:	012345
Študijný program:	jadrové a fyzikálne inžinierstvo
Študijný odbor:	elektrotechnika
Vedúci práce:	tituly, Meno Priezvisko, tituly
Vedúci pracoviska:	tituly Meno Priezvisko, tituly
Miesto vypracovania:	Ústav jadrového a fyzikálneho inžinierstva

Názov práce: **Rozšírená šablóna záverečnej práce na FEI STU v Bratislave v systéme LaTeX**

Jazyk, v ktorom sa práca vypracuje: slovenský jazyk

Špecifikácia zadania:

1. Vytvorte šablónu záverečnej práce pre študentov bakalárskeho a inžinierskeho štúdia na FEI STU v Bratislave.
2. Napíšte návod na písanie záverečnej práce s použitím vytvorenej šablóny.
3. Vytvorte prehľad najpoužívanejších typov citovaných zdrojov a zdokumentujte ich tak, aby boli v súlade s normou ISO 690.

Zoznam odbornej literatúry:

1. Zákon č. 131/2002 Z. z. z 21. februára 2002 o vysokých školách a o zmene a doplnení niektorých zákonov [online]. [cit. 2024-09-01]. Dostupné na https://www.slov-lex.sk/static/pdf/2002/131/ZZ_2002_131_20240901.pdf
2. Vyhláška č. 233/2011 Z. z. Ministerstva školstva, vedy, výskumu a športu Slovenskej republiky z 1. júla 2011, ktorou sa vykonávajú niektoré ustanovenia zákona č. 131/2002 Z. z. o vysokých školách a o zmene a doplnení niektorých zákonov v znení neskorších predpisov [online]. [cit. 2024-09-01]. Dostupné na https://www.slov-lex.sk/static/pdf/2011/233/ZZ_2011_233_20201015.pdf

3. Metodické usmernenie Ministerstva školstva, vedy, výskumu a športu SR č. 56/2011 o náležitostiach záverečných prác, ich bibliografickej registrácii, kontrole originality, uchovávaní a sprístupňovaní č. 2011-11513/31015:4-071 [online]. [cit. 2024-09-24]. Dostupné na http://www.minedu.sk/metodicke-usmernenie-c-562011-o-nalezitostiach-zaverecnych-prac-ich-bibliografickej-registracii-uchovavani-a-spristupnovani/.
4. STN ISO 214: 1998. Dokumentácia. Abstrakty (referáty) pre publikácie a dokumentáciu. Bratislava: Slovenský ústav technickej normalizácie. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
5. STN ISO 690: 2022. Dokumentácia – Bibliografické odkazy – Obsah, forma a štruktúra. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
6. STN 01 6910: 2023. Pravidlá písania a úpravy písomností. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
7. STN ISO 2145: 1997. Dokumentácia. Číslovanie oddielov a pododdielov písaných dokumentov. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
8. STN ISO EN 80 000-1: 2022. Veličiny a jednotky. Bratislava. Úrad pre metrológiu a skúšobníctvo Slovenskej republiky.
9. LICHNEROVÁ, L. a B. BELLÉROVÁ. Nové pravidlá citovania podľa ISO 690 z roku 2021. *ITlib. Informačné technológie a knižnice,* 3–4/2023, 10 – 22. [cit. 2024-09-06]. Dostupné na http://doi.org/10.52036/1335793X.2023.3-4.10-22.
10. HOFTICH, M. ISO 690 biblatex style [online]. [cit. 2024-09-30]. Dostupné na https://mirrors.ibiblio.org/CTAN/macros/latex/contrib/biblatex-contrib/biblatex--iso690/biblatex-iso690.pdf.

Termín odovzdania bakalárskej práce:	31. 05. 2025
Dátum schválenia zadania bakalárskej práce:	28. 02. 2025
Zadanie bakalárskej práce schválil:	tituly Meno Priezvisko, tituly – garant študijného programu

Poďakovanie

Týmto by som sa chcel úprimne poďakovať vedúcemu mojej diplomovej práce Ing. Marošovi Čavojskému, PhD. za jeho cenné rady, pomoc a vedenie počas písania tejto práce. Moje poďakovanie patrí taktiež mojej rodine za správnu výchovu, potrebunú k dopraqcovaniu až sem,mojej priateľke za psychickú podporu a poskytnutie príjemného prostredia pri písaní práce, ako aj Ministerstvu školstva za bezplatnú možnosť študovať na tejto fakulte.

Abstrakt

Táto diplomová práca sa zameriava na vývoj aplikácie, ktorá používateľom umožňuje jednoduché sociálne prepájanie prostredníctvom organizovania a vyhľadávania podujatí na základe ich polohy. Aplikácia podporuje vytváranie menších neformálnych stretnutí jednotlivcov so spoločnými záujmami, ako aj organizovanie väčších podujatí zo strany firiem a organizácií.

Cieľom práce bolo vytvoriť používateľsky prívetivú aplikáciu využívajúcu moderné postupy a technológie, ktorá zjednodušuje interakciu medzi používateľmi so spoločnými záujmami a podporuje živé stretnutia v reálnom čase.

Systém je navrhnutý ako Android aplikácia využívajúca GPS technológiu na zistenie polohy používateľa a poskytuje možnosti pre vytváranie a správu udalostí, pozývanie účastníkov a správu notifikácií. Aplikácia funguje aj na pozadí a je schopná posielať notifikácie. Využíva Firebase databázu pre ukladanie a synchronizáciu dát v reálnom čase, ako aj vlastný odporúčací systém pre navrhovanie udalostí na základe histórie používateľa a jeho záujmov. Aplikácia má taktiež implementovaný systém na detekciu nevhodných výrazov v názvoch a popisoch podujatí.

Výsledná aplikácia bola otestovaná na reálnych zariadeniach a hodnotená z hľadiska funkčnosti a použiteľnosti.

Kľúčové slová

záverečná práca, šablóna, LaTeX, formátovanie textu, citácie

Abstract

The manual for students of the Faculty of Electrical Engineering and Information Technology at Slovak University of Technology in Bratislava provides advice and guidelines on how to approach the formal aspects of writing a final thesis for university studies. The document can also serve as a template for the thesis in the LaTeX typesetting system. It covers in detail the rules for typesetting mathematical equations, numbering floating objects, and referencing them. It also thoroughly addresses the method of citing external literary sources.

Keywords

Final thesis, events, meeting, organizer, location, gps, kotlin, android

Obsah

Úvod **13**

1 Štruktúra záverečnej práce **14**

1.1 Základné pojmy 14

1.2 Architektúra a komunikácia v distribuovaných systémoch 14

1.2.1 Klient-server architektúra 14

1.2.2 Komunikačné rozhrania a protokoly (REST API a HTTP) . . . 14

1.2.3 Cloudové riešenia a služby 15

1.3 Správa a ukladanie dát 15

1.3.1 Relačné databázy (SQL) 16

1.3.2 Nerelačné databázy (NoSQL) 16

1.3.3 Real-time databázy 16

1.3.4 Autentifikácia 16

1.3.5 Faktory a metódy autentifikácie 16

1.3.6 Digitálna autentifikácia a bezpečnosť 17

1.3.7 Autorizácia 17

1.3.8 Mechanizmus trvalého prihlásenia (Token-based Authentication) 17

1.4 Vývoj mobilných aplikácií 18

1.4.1 Mobilná aplikácia 18

1.4.2 Programovacie jazyky 18

1.4.3 Softvérové frameworky 18

1.4.4 Vývojové prostredia a proces spracovania kódu 18

1.4.5 Prístupy k vývoju mobilných aplikácií 19

1.4.6 Technické obmedzenia a optimalizácia výkonu 19

1.4.7 Distribúcia a ekosystém platforiem 20

1.5 Geolokačné technológie a ich využitie 20

1.5.1 Operačná efektivita a hybridný prístup 21

1.5.2 Matematické metódy určovania polohy 21

1.6 Technológia QR kódov 21

1.6.1 Kľúčové vlastnosti a štruktúra 22

1.6.2 Využitie v mobilných zariadeniach 22

1.7 Definícia a význam odporúčacích systémov 22

1.7.1 Obsahové filtrovanie (Content-Based Filtering) 23

1.7.2 Kolaboratívne filtrovanie (Collaborative Filtering) 23

1.7.3 Hybridné systémy 24

1.8 Techniky spracovania dát v odporúčacích systémoch 25

1.8.1 Dizajn a používateľská skúsenosť (UX/UI) 25
1.9 Architektonické vzory a princípy návrhu 25
1.10 Základné princípy moderného návrhu 26
1.11 Porovnanie architektonických vzorov 26

2 Súčasný stav riešenej problematiky 27
2.1 Facebook Events 27
2.2 Meetup 28
2.3 AllEvents.in 29
2.4 Hapsy 30
2.5 Mapibara 31
2.6 Zhrnutie analýzy súčasného stavu 32
2.7 Syntéza poznatkov a východiská pre vlastný návrh 33

3 Použité technológie 35
3.1 Vývojové prostredie a platforma 35
3.1.1 Operačný systém Android 35
3.1.2 Distribúcia a ekosystém platforiem 35
3.2 Programovací jazyk Kotlin 35
3.3 UI Toolkit: Jetpack Compose 36
3.4 Vývojové nástroje 36
3.5 Technická špecifikácia a použité knižnice 36
3.6 Architektúra projektu 36
3.6.1 Moderný prístup: MVVM v kombinácii s Jetpack Compose . . . 37
3.7 Platforma Firebase 38
3.7.1 Dôvody voľby a kľúčové výhody 38
3.8 Firebase Authentication 38
3.9 Firebase Firestore 39
3.10 Firebase Storage 39
3.11 Firebase 39
3.12 Firebase Authentication 39
3.13 Firebase Firestore 39
3.14 Firebase Storage 40
3.15 Definícia a význam odporúčacích systémov 40
3.15.1 Obsahové filtrovanie (Content-Based Filtering) 40
3.15.2 Kolaboratívne filtrovanie (Collaborative Filtering) 40
3.15.3 Hybridné systémy 41
3.16 Techniky spracovania dát v odporúčacích systémoch 41

3.17 Metriky hodnotenia kvality odporúčaní 42
3.18 Google Maps Platform 43
3.18.1 Maps SDK for Android 44
3.18.2 Places SDK for Android 44

4 Návrh riešenia **45**
4.0.1 Funkcionálne a Nefunkcionálne požiadavky 45
4.0.2 Funkcionálne požiadavky 45
4.0.3 Nefunkcionálne požiadavky 47
4.0.4 UML diagramy 48

Záver **52**

Literatúra **54**
Použitie nástrojov umelej inteligencie 54

A Algoritmus **55**

B Výpis dlhého kódu **56**

C Slovníček pojmov **60**

Zoznam značiek a skratiek

$\vec{a}, \boldsymbol{a}$ – zrýchlenie (m/s^2)

a – veľkosť zrýchlenia (m/s^2), mriežková konštanta (nm)

AC – striedavý prúd, z angl. *Accelerating Current*

AFM – mikroskop atómových síl, z angl. *Atomic Force Microscope*

AI – umelá inteligencia, z angl. *Artificial Intelligence*

ε_0, ε_0 – elektrická konštanta ($\varepsilon_0 = 8{,}85418782\,.\,10^{-12}\,\mathrm{m^{-3}kg^{-1}s^4A^2}$)

GAI – generatívna umelá inteligencia

λ – vlnová dĺžka svetla (nm)

LED – svetlo emitujúca dióda, z angl. *Light-Emitting Diode*

LLM – veľký jazykový model, z angl. *Large Language Model*

$\vec{F}, \boldsymbol{F}, F$ – vektor sily, veľkosť sily (N)

q – elektrický náboj (C)

$\boldsymbol{r}$, r – polohový vektor, veľkosť polohového vektora (m)

SEM – skenovací elektrónový mikroskop

TEM – transmisný elektrónový mikroskop

U – elektrické napätie (V)

Zoznam algoritmov

1 Vypočítaj $y = x^n$ 55

Zoznam výpisov kódov

1 Ukážka výpisu kódu programu načítaného z externého súboru pomocou makra `\lstinputlisting` 56

Úvod

Digitalizácia priniesla veľa výhod: online nákupy, objednanie jedla jedným klikom, stretnutia cez internet bez fyzickej prítomnosti druhej osoby a mnoho ďalších vecí, ktoré nám uľahčujú život. Ale nájdu sa v tomto svete aj nevýhody?

Čím viac nás vtiahne tento virtuálny svet, tým viac zabúdame na to, čo je realita. Nakoniec zistíme, že nekonečné scrollovanie na sociálnych sieťach nám neprinesie do života dlhodobú radosť, skôr nám ho urobí nudnejším. Robí nás pasívnejšími a závislejšími od technológií. Talenty, ktoré sme možno kedysi mali, sa postupne vytrácajú. A aj keby sme mali chvíľkovú potrebu s tým niečo spraviť, zabrzdí nás fakt, že tomu musíme venovať úsilie a čas.

Ale čo ak by existovala jednoduchá aplikácia, kde si človek môže nájsť ľudí so spoločnými záujmami, akcie, ktoré sa mu páčia, alebo objaviť nové hobby len preto, lebo vyzerajú zaujímavo na fotkách? Toto je dôvod, prečo som sa rozhodol vytvoriť túto mobilnú aplikáciu.

Cieľom tejto diplomovej práce je vytvoriť mobilnú aplikáciu pre systém Android, ktorá spája ľudí, pomáha organizátorom jednoducho vytvárať vlastné podujatia a pomáha používateľom nájsť zábavu, nové záujmy alebo len tak tráviť čas s priateľmi. Pri tejto práci využijeme množstvo postupov a technológií, ktoré nám umožnia vytvoriť funkčnú a používateľsky príjemnú aplikáciu.

V práci sa budeme venovať návrhu a implementácii tejto aplikácie. Využijeme databázy na ukladanie informácií o používateľoch a podujatiach a navrhneme odporúčací systém vzhľadom na používateľovu históriu. Taktiež navrhneme systém, ako zamedziť zneužívaniu aplikácie a ako zabezpečiť kontrolu nevhodného obsahu. Aplikácia bude mať jednoduché a intuitívne rozhranie, ktoré umožní používateľom ľahko sa orientovať a využívať všetky funkcie. Nakoniec budeme aplikáciu testovať a získavať spätnú väzbu od používateľov, aby sme ju mohli vylepšiť a prispôsobiť ich potrebám. Verím, že táto aplikácia bude užitočná pre mnoho ľudí, pomôže im nájsť nové záujmy a že jej finálna verzia prinesie do tohto sveta hodnotu.

1 Štruktúra záverečnej práce

Táto diplomová práca je rozdelená na niekoľko logických celkov, ktoré postupne opisujú cestu od teoretickej analýzy až po finálnu implementáciu a testovanie aplikácie.

1.1 Základné pojmy

V tejto kapitole sa zameriame na všeobecné pojmy a technológie, ktoré sú kľúčové pre správne pochopenie problematiky vývoja mobilných aplikácií a implementácie odporúčacích systémov. Cieľom je poskytnúť čitateľovi pevný základ, na ktorom bude možné stavať v ďalších častiach práce.

1.2 Architektúra a komunikácia v distribuovaných systémoch

Základom modernej sieťovej komunikácie je efektívne rozdelenie úloh medzi jednotlivé uzly v sieti. Tento prístup umožňuje lepšiu správu zdrojov a vysokú škálovateľnosť aplikácií.

1.2.1 Klient-server architektúra

Architektúra klient-server predstavuje systém, ktorý vykonáva funkcie klienta aj servera s cieľom podporiť zdieľanie informácií medzi nimi. Tento model umožňuje viacerým používateľom pristupovať k rovnakej databáze v rovnakom čase, pričom databáza slúži na ukladanie veľkého množstva informácií [**sulyman2014**]. (Zdroj: `https://www.researchgate.net/profile/Shakirat-Sulyman/publication/271295146_Client-Server_Model/links/5864e11308ae8fce490c1b01/Client-Server-Model.pdf`)

- **Server:** Predstavuje počítač alebo softvérový systém, ktorý poskytuje dáta, zdroje alebo služby iným počítačom v sieti, ktoré označujeme ako klienti.
  (Zdroj: `https://en.wikipedia.org/wiki/Server_(computing)`)
- **Klient:** Ide o kus počítačového hardvéru alebo softvéru, ktorý pristupuje k službe sprístupnenej serverom v rámci modelu klient-server v počítačových sieťach.
  (Zdroj: `https://en.wikipedia.org/wiki/Client_(computing)`)

1.2.2 Komunikačné rozhrania a protokoly (REST API a HTTP)

Na to, aby si klient a server v distribuovanom systéme rozumeli, potrebujú spoločný komunikačný štýl a protokol. V súčasnosti je najrozšírenejším štandardom rozhranie Representational State Transfer (REST API). Ide o aplikačné programové rozhranie,

ktoré vyhovuje princípom architektonického štýlu REST. Jeho podstata spočíva v práci so zdrojmi, ktorými môže byť akákoľvek informácia, napríklad používateľ, produkt alebo dokument [**google_cloud_rest**].

Celý proces komunikácie prebieha v dvoch základných krokoch:

1. **Klient odošle požiadavku:** Mobilná aplikácia iniciuje požiadavku na vykonanie akcie smerom k API. Táto požiadavka obsahuje metódu HTTP, hlavičky s metadátami a prípadne telo s dátami.

2. **Server spracuje požiadavku:** Server prijme a overí požiadavku (autentifikácia a autorizácia). Následne ju spracuje, čo môže zahŕňať načítanie dát z databázy alebo vytvorenie nového zdroja.
   (Zdroj: `https://cloud.google.com/discover/what-is-rest-api`)

Samotným nosným médiom pre túto komunikáciu je protokol Hypertext Transfer Protocol (HTTP). Ide o základný aplikačný protokol, ktorý umožňuje komunikáciu a prenos dát v prostredí celosvetovej siete. V architektúre klient-server funguje ako „jazyk“, ktorým sa tieto dve strany dorozumievajú.
(Zdroj: `https://gemini.google.com/app/6ed6e44697b48104` – konzultácia 29.03.2026)

1.2.3 Cloudové riešenia a služby

Cloud môžeme prirovnať k virtuálnemu priestoru, prostredníctvom ktorého používatelia vytvárajú, zdieľajú a narábajú s obrovským množstvom informácií rôzneho typu. Ide o výpočtové zdroje dostupné na vyžiadanie (on-demand), ku ktorým má používateľ prístup cez internet [**eset_cloud**].

Dáta sa ukladajú na cloudové servery umiestnené v dátových centrách po celom svete a sú plne riadené ich poskytovateľom. Za poplatok môže zákazník na týchto serveroch prevádzkovať svoj softvér, využívať ich výpočtový výkon či úložisko na ukladanie vlastných dát.
(Zdroj: `https://bezpecnenanete.eset.com/sk/it-bezpecnost/co-je-to-cloud-a-ako-funguje/`)

1.3 Správa a ukladanie dát

Databáza funguje ako digitálne úložisko, ktoré organizuje dáta potrebné pre chod aplikácie. Podľa spôsobu ukladania a výkonu ich najčastejšie delíme na:

1.3.1 Relačné databázy (SQL)

Dáta sú v tomto modeli organizované do **tabuliek** s riadkami a stĺpcami. Medzi najpoužívanejšie systémy podľa portálu Webglobe [**webglobe_databaza**] patria:

- **MySQL:** Jednoduchá a rýchla databáza, hojne využívaná pri menších až stredne veľkých aplikáciách.
- **MariaDB:** Ponúka vyšší výkon a rozšírené funkcie, je vhodná aj pre náročnejšie aplikácie.
- **PostgreSQL:** Výkonná a robustná databáza s pokročilými funkciami, ideálna pre veľké a zložité aplikácie.

(Zdroj: `https://www.webglobe.sk/poradna/co-je-databaza`)

1.3.2 Nerelačné databázy (NoSQL)

Na rozdiel od tabuliek ukladajú dáta flexibilne, najčastejšie ako **dokumenty**. Tento typ (napr. Firebase Firestore) je kľúčový pre moderné mobilné aplikácie, pretože umožňuje meniť štruktúru dát „za pochodu“ bez nutnosti reštartovania systému [**aws_nosql**].
(Zdroj: `https://aws.amazon.com/nosql/`)

1.3.3 Real-time databázy

Špeciálny typ databázy, ktorý **okamžite synchronizuje zmeny** medzi serverom a všetkými pripojenými používateľmi bez nutnosti obnovovania aplikácie. Je to základ pre funkcie ako chaty alebo živé aktualizácie obsahu [**firebase_db**].
(Zdroj: `https://firebase.google.com/docs/database`)

1.3.4 Autentifikácia

Autentifikácia je proces overenia používateľa (overenia jeho identity), teda zistenie, o koho ide. Môže sa to overiť podľa kombinácie mena a hesla, biometrie (napr. otlačku prsta alebo rozpoznania hlasu), alebo iným zariadením.

https://sk.wikipedia.org/wiki/Autentifik

1.3.5 Faktory a metódy autentifikácie

V informatike delíme spôsoby overenia používateľa do troch základných kategórií podľa toho, na čom sú založené:

- **Znalostný faktor:** Niečo, čo používateľ pozná (napr. heslo, PIN).

- **Faktor vlastníctva:** Niečo, čo používateľ má (napr. smartfón, bezpečnostný token).
- **Dedičný faktor:** Niečo, čím používateľ je (biometria – odtlačok prsta, sken tváre).

Pre dosiahnutie vysokej bezpečnosti sa odporúča **viacfaktorové overenie** (MFA), ktoré kombinuje aspoň dva z vyššie uvedených faktorov. V mobilných zariadeniach sa čoraz častejšie využíva aj tzv. **kontinuálne overovanie**, ktoré neustále monitoruje identitu používateľa na základe biometrie správania (napr. štýl písania alebo dynamika dotyku).

1.3.6 Digitálna autentifikácia a bezpečnosť

Proces digitálneho overovania pozostáva z prihlásenia (vytvorenie poverení) a samotnej autentifikácie (predloženie dôkazu identity, napr. tokenu). Pri sieťovej komunikácii je kľúčové chrániť tento proces pred útokmi typu *Man-in-the-Middle* (MITM), kedy sa útočník snaží zachytiť prenášané údaje.
(Zdroj: `https://sk.wikipedia.org/wiki/AutentifikÃącia`)

1.3.7 Autorizácia

Autorizácia je proces overenia oprávnení pre danú úlohu, teda či autentifikovaný používateľ (systém) má oprávnenie vykonať úlohu, o ktorú žiada (napr. vstúpiť do systému alebo vykonať platobnú operáciu).

1.3.8 Mechanizmus trvalého prihlásenia (Token-based Authentication)

V moderných mobilných aplikáciách je žiaduce, aby používateľ nebol nútený zadávať prihlasovacie údaje pri každom otvorení aplikácie. Tento proces sa nazýva *perzistentná autentifikácia* a je realizovaný pomocou systému dvoch typov tokenov (bezpečnostných kľúčov):

- **Access Token (Prístupový kľúč):** Krátkodobý token (platnosť zvyčajne 60 minút), ktorý mobilná aplikácia pripája ku každej požiadavke na server. Slúži ako dôkaz, že používateľ je momentálne overený.
- **Refresh Token (Obnovovací kľúč):** Dlhodobý token uložený v šifrovanej pamäti zariadenia. Jeho jedinou úlohou je vyžiadať si nový *Access Token* v prípade, že pôvodný vypršal.

Tento cyklus prebieha na pozadí bez zásahu používateľa (tzv. *Silent Re-authentication*). Ak je *Refresh Token* stále platný, aplikácia automaticky obnoví reláciu, čím zabezpečí plynulý používateľský zážitok pri zachovaní vysokej bezpečnosti, keďže samotné heslo sa sieťou prenáša iba raz – pri prvotnom prihlásení.
(Zdroj: `https://auth0.com/docs/secure/tokens/refresh-tokens`)

1.4 Vývoj mobilných aplikácií

1.4.1 Mobilná aplikácia

Mobilná aplikácia je softvérová aplikácia vytvorená špecificky pre smartfóny, tablety alebo iné mobilné zariadenia, ktorá si vyžaduje inštaláciu, čím sa líši od bežného webu. V prvom rade vďaka nim dokážeme rozlíšiť smartfón od bežného telefónu.
(Zdroj: `https://www.eliteml.sk/blog/mobilne-aplikacie-v-skratke/`)

1.4.2 Programovacie jazyky

Programovací jazyk obsahuje inštrukcie vytvorené programátormi, ktoré riadia počítač pri vykonávaní konkrétnych úloh. Tieto inštrukcie, bežne vnímané ako komplexný kód, dodržiavajú odlišnú syntax špecifickú pre každý programovací jazyk a slúžia ako nevyhnutné nástroje pre vývoj softvéru a rôzne výpočtové úlohy.
(Zdroj: `https://codelabsacademy.com/sk/blog/what-is-a-programming-language`)

1.4.3 Softvérové frameworky

Framework je softvérová platforma alebo sada nástrojov, ktorá poskytuje štruktúru a základné funkcionality pre vývoj softvéru. Frameworky uľahčujú vývoj aplikácií tým, že poskytujú preddefinovanú štruktúru, architektúru a sadu nástrojov, ktoré zjednodušujú a urýchľujú proces vývoja.
(Zdroj: `https://coderama.com/slovnik/framework`)

1.4.4 Vývojové prostredia a proces spracovania kódu

Na písanie a správu programovacieho kódu využívajú vývojári špecializované softvérové nástroje, ktoré označujeme ako **IDE** (*Integrated Development Environment* – Integrované vývojové prostredie). Tieto nástroje siahajú od jednoduchých textových editorov s podporou syntaxe až po komplexné systémy, ktoré obsahujú:

- **Editor kódu:** Miesto na samotný zápis inštrukcií s funkciami ako automatické dopĺňanie kódu.

- **Debugger:** Nástroj na vyhľadávanie a opravu logických chýb v programe.
- **Kompilátor / Interpret:** Špeciálny program, ktorý transformuje (kompiluje) kód napísaný programátorom do binárnej podoby (strojového kódu), ktorej rozumie hardvér mobilného zariadenia alebo procesor.

Medzi najpoužívanejšie prostredia pre mobilný vývoj patria napríklad **Android Studio**, **Visual Studio Code** (populárne pre frameworky ako React Native alebo Flutter) či **Xcode** pre platformu iOS. Výber konkrétneho prostredia často závisí od zvoleného programovacieho jazyka a cieľovej platformy.
(Zdroj: `https://it-slovnik.cz/pojem/ide-integrovane-vyvojove-prostredi`)

1.4.5 Prístupy k vývoju mobilných aplikácií

Podľa spoločnosti IBM možno proces vývoja mobilných aplikácií definovať ako tvorbu softvéru navrhnutého špecificky pre mobilné koncové zariadenia, pričom kľúčovým rozhodnutím je technológia implementácie. V súčasnosti sa vývojári rozhodujú primárne medzi natívnym a multiplatformovým prístupom [**pullur_medium_2025**]:

- **Natívne aplikácie:** Sú vyvíjané priamo pre konkrétnu platformu (Android, iOS) pomocou dedikovaných nástrojov. Ponúkajú najvyšší výkon a neobmedzený prístup k hardvéru.
- **Webové mobilné aplikácie:** Ide o responzívne webové stránky využívajúce technológie HTML5, CSS a JavaScript. Modernou formou sú tzv. *Progressive Web Apps* (PWA).
- **Hybridné aplikácie:** Kombinujú predchádzajúce prístupy. Jadro aplikácie je písané vo webových technológiách, ktoré sú následne "zabalené"do natívneho kontajnera.

(Zdroj: `https://opac.crzp.sk/?fn=detailBiblioFormChild01BU&sid=9A7D98113A4E7D6F09AF71A0EB46&seo=CRZP-detail-kniha`)

1.4.6 Technické obmedzenia a optimalizácia výkonu

Vývoj pre mobilné platformy vyžaduje špecifický prístup k správe prostriedkov, ktoré sú v porovnaní s desktopovými systémami limitované:

1. **Energetická a systémová efektivita:** Natívne aplikácie vykazujú vyššiu energetickú efektivitu, čo je kľúčové pri aplikáciách využívajúcich lokalizačné služby (GPS) [**pullur_medium_2025**].

2. **Cloudové spracovanie (*Cloud Offloading*):** Náročné výpočtové operácie, ako sú komplexné odporúčacie algoritmy, je vhodné delegovať na cloudové servery (napr. Firebase).

3. **Odozva používateľského rozhrania (UI):** Natívne frameworky dosahujú najlepšiu plynulosť animácií a rýchlosť odozvy na dotyk.

1.4.7 Distribúcia a ekosystém platforiem

Platforma Android, s približne 70 % trhovým podielom, je často preferovaná pre nasadenie MVP kvôli flexibilnejším pravidlám. Naopak, ekosystém iOS vyžaduje striktnejšie dodržiavanie štandardov. Multiplatformové nástroje ako Flutter a React Native umožňujú súčasné nasadenie na oboch platformách s určitým kompromisom v oblasti výkonu. (Zdroj: `https://medium.com/@riyaspullurofficial/native-vs-cross-platform-app-size-and-performance-comparison-550cb7f918b0`)

Tabuľka 1: Súhrnné porovnanie vývojových prístupov (podľa [**pullur_medium_2025**])

Metrika	**Native**	**Flutter**	**React Native**	**KMM**
Veľkosť aplikácie	**Najmenšia**	Väčšia	Väčšia	Stredná
Zdieľanie kódu	Žiadne	95 % +	90 % +	Logika
Spotreba batérie	**Nízka**	Stredná	Vysoká	Nízka

1.5 Geolokačné technológie a ich využitie

Geolokácia je proces identifikácie geografickej polohy zariadenia. Podľa analýzy spoločnosti Sensolus [**sensolus_geoloc**] sa v moderných systémoch využívajú štyri hlavné technológie, ktoré sa líšia presnosťou a energetickou náročnosťou:

- **GPS (Global Positioning System):** Využíva satelitné signály. Je najpresnejší v exteriéri (na metre), ale má vysokú spotrebu batérie a nefunguje vo vnútri budov.

- **Wi-Fi Sniffing:** Určuje polohu skenovaním okolitých Wi-Fi sietí. Je ideálny pre mestské prostredie a prechod medzi budovami, pričom spotrebúva menej energie ako GPS.

- **BLE (Bluetooth Low Energy):** Využíva malé vysielače (beacons). Poskytuje extrémnu presnosť v interiéroch (napr. sklady), ale vyžaduje inštaláciu fyzických zariadení v priestore.

- **LPWAN Geolocation:** Využíva nízkoenergetické siete (napr. Sigfox) na trianguláciu signálu. Má najdlhšiu výdrž batérie (roky), ale nižšiu presnosť (stovky metrov).

1.5.1 Operačná efektivita a hybridný prístup

Výber správnej technológie priamo ovplyvňuje prevádzkovú efektivitu systému. V mobilnom vývoji sa najčastejšie využíva tzv. **hybridná geolokácia**, ktorá inteligentne prepína medzi GPS a Wi-Fi podľa dostupnosti signálu a stavu batérie. To umožňuje aplikáciám efektívne sledovať polohu používateľa bez nadmerného zaťaženia hardvéru. (Zdroj: `https://www.sensolus.com/knowledge-hub/behind-the-tech/four-geolocation-technologies-compared-how-can-they-improve-your-operational-efficiency/`)

1.5.2 Matematické metódy určovania polohy

Na výpočet presnej polohy zariadenia zo získaných signálov (rádiových, satelitných či Bluetooth) sa v informatike využívajú nasledujúce geometrické a fyzikálne metódy:

- **TOA (Time of Arrival):** Metóda meria absolútny čas, za ktorý signál prejde od vysielača k prijímaču. Na určenie polohy v 2D priestore sú potrebné aspoň tri vysielače (trilaterácia).
- **TDOA (Time Difference of Arrival):** Nameria sa rozdiel v čase príchodu signálov z viacerých vysielačov. Je to praktickejšie ako TOA, pretože nevyžaduje dokonalú synchronizáciu hodín medzi vysielačom a mobilným zariadením.
- **AOA (Angle of Arrival):** Táto metóda neurčuje vzdialenosť, ale **uhol**, pod ktorým signál prichádza k anténe. Kombináciou uhlov z dvoch staníc je možné určiť presný bod polohy (triangulácia).
- **RSS (Received Signal Strength):** Odhaduje vzdialenosť na základe útlmu sily signálu. Čím je signál slabší, tým je zariadenie pravdepodobne ďalej od vysielača (často využívané pri Wi-Fi a Bluetooth).

Tieto metódy sú matematickým základom pre fungovanie GPS (prevažne TOA/TDOA) aj moderných systémov pre lokalizáciu v interiéroch.

1.6 Technológia QR kódov

QR kód (*Quick Response Code*) je dvojrozmerný čiarový kód, ktorý v roku 1994 vyvinula spoločnosť Denso Wave (patriaca do skupiny Toyota). V roku 2000 bol schválený ako

medzinárodný štandard ISO (ISO/IEC18004). Pôvodne bol navrhnutý pre sledovanie dielov v automobilovom priemysle, no vďaka svojej efektivite sa rozšíril do všetkých oblastí digitálnej komunikácie.

1.6.1 Kľúčové vlastnosti a štruktúra

QR kód sa od klasických lineárnych čiarových kódov líši niekoľkými zásadnými charakteristikami:

- **Vysoká hustota dát:** Dokáže uchovať oveľa väčšie množstvo informácií na malej ploche (text, URL adresy, kontaktné údaje).
- **Podpora znakov:** Okrem číslic a latinky podporuje aj zložitejšie znakové sady (napr. binárne dáta alebo Kanji).
- **Odolnosť voči poškodeniu:** Vďaka vstavanej korekcii chýb (Reed-Solomonov kód) je možné QR kód správne prečítať, aj keď je časť jeho plochy znečistená alebo poškodená.
- **Otvorený štandard:** Patent na túto technológiu bol uvoľnený pre verejnosť, čo znamená, že QR kódy môže generovať a používať ktokoľvek bezplatne.

1.6.2 Využitie v mobilných zariadeniach

V súčasnosti je väčšina moderných smartfónov vybavená kamerami s natívnou podporou čítania QR kódov. Tento mechanizmus sa najčastejšie využíva na:

- **Rýchly prístup k informáciám:** Automatické presmerovanie prehliadača na zakódovanú URL adresu.
- **Bezkontaktné interakcie:** Elektronické vstupenky, platby (napr. skenovanie faktúr) alebo rýchle pripájanie k Wi-Fi sieťam.
- **Autentifikáciu:** Prepojenie mobilnej aplikácie s webovým rozhraním (napr. WhatsApp Web).

(Zdroj: `https://www.iso.org/obp/ui/en/#iso:std:iso-iec:18004:ed-4:v1:en`)

1.7 Definícia a význam odporúčacích systémov

S prudkým rozvojom internetových technológií a inteligentných mobilných zariadení čelí moderný používateľ fenoménu informačného preťaženia. V prostredí s takmer

neobmedzeným množstvom digitálneho obsahu a služieb sa odporúčacie systémy (angl. *Recommender Systems*) stali kľúčovým nástrojom na personalizovanú filtráciu informácií.

Ich primárnou úlohou je predpovedať preferencie používateľa a zredukovať rozsiahly priestor dostupných položiek na úzky výber relevantných možností, čím sa zvyšuje celková spokojnosť a efektivita interakcie s platformou. Súčasný výskum sa čoraz viac orientuje na spracovanie implicitných dát. Ide o behaviorálne vzorce, ako sú kliknutia (*click-stream*), história navštívených miest či čas strávený interakciou s konkrétnym objektom [**electronics11010141**].

1.7.1 Obsahové filtrovanie (Content-Based Filtering)

Tento prístup je založený na analýze špecifických atribútov položiek. Systém identifikuje vlastnosti objektov (napr. textové informácie, kategórie alebo tagy), o ktoré používateľ v minulosti prejavil záujem, a na ich základe generuje odporúčania pre podobné položky [**electronics11010141**].

- **Princíp:** Analýza atribútov a metadát položiek (napr. žáner, kategória) s cieľom nájsť podobnosť s preferenciami používateľa.
- **Využitie:** Efektívne najmä pri službách pracujúcich s textovými dátami, hudbou alebo e-commerce [**electronics11010141**].
- **Limity:** Nízka miera diverzity (angl. *serendipity*) – používateľovi nie sú ponúkané nové, neobjavené kategórie obsahu, čo vedie k efektu informačnej bubliny.

1.7.2 Kolaboratívne filtrovanie (Collaborative Filtering)

Kolaboratívne filtrovanie predpovedá vkus používateľa na základe analýzy správania a hodnotení celej komunity používateľov (tzv. „wisdom of the crowd“).

- **Memory-based:** Hľadá podobnosť priamo medzi používateľmi (*User-based*) alebo položkami (*Item-based*) pomocou algoritmov ako KNN alebo korelačných koeficientov.
- **Model-based:** Využíva pokročilé štatistické metódy a techniky strojového učenia, ako sú zhlukovanie (*Clustering*), *Singular Value Decomposition* (SVD) alebo *Principal Component Analysis* (PCA).
- **Hlavné nedostatky:** Zdroj [**electronics11010141**] definuje tri kritické obmedzenia:
  - *Cold Start:* Problém s odporúčaním položiek pre nových používateľov bez histórie interakcií.

- *Sparsity (Riedkosť dát):* Situácia, keď je v systéme príliš málo hodnotení na to, aby bolo možné spoľahlivo vypočítať podobnosť.
- *Gray Sheep (Sivé ovce):* Používatelia s veľmi unikátnym vkusom, ktorí nevykazujú podobnosť so žiadnou väčšou skupinou používateľov, čo sťažuje generovanie odporúčaní.

1.7.3 Hybridné systémy

Hybridné modely kombinujú viaceré techniky s cieľom eliminovať slabiny jednotlivých prístupov, najmä problém riedkosti dát (*sparsity*) a studeného štartu. Podľa Burkeho klasifikácie [**electronics11010141**] existuje sedem základných metód hybridizácie, ktoré sú uvedené v tabuľke 5. Hlavným cieľom väčšiny štúdií zaoberajúcich sa hybridnými

Tabuľka 2: Typy hybridných odporúčacích systémov podľa Burkeho [**electronics11010141**]

Metóda	**Princíp fungovania**
Weighted	Skóre z rôznych techník sa kombinuje pomocou numerických váh.
Switching	Systém prepína medzi technikami v závislosti od kontextu.
Mixed	Odporúčania z viacerých zdrojov sú zobrazené súčasne.
Feature Combination	Atribúty z jedného zdroja slúžia ako vstupné črty pre druhý model.
Feature Augmentation	Výstup jednej techniky sa stáva parametrom pre ďalšie spracovanie.
Cascade	Postupné zjemňovanie poradia (jeden systém vytvorí rebríček, druhý ho upresní).
Meta-level	Celý model vygenerovaný jednou technikou slúži ako vstup pre druhú.

modelmi po roku 2010 je integrácia tzv. vedľajších informácií (*side information*), ako sú sociálne väzby medzi používateľmi alebo kontextové dáta, ktoré dopĺňajú chýbajúce explicitné hodnotenia. Týmto spôsobom hybridné systémy dosahujú vyššiu presnosť a robustnosť v porovnaní so samostatnými modelmi.

1.8 Techniky spracovania dát v odporúčacích systémoch

Pre efektívne fungovanie vyššie spomínaných modelov sa využívajú špecifické techniky dolovania dát (Data Mining):

1. **Text Mining:** Analýza neštruktúrovaných textov pomocou sémantickej analýzy a váhovania pojmov (napr. TF-IDF). Slúži na pochopenie obsahu podujatí a preferencií používateľa.
2. **K-Nearest Neighbor (KNN):** Algoritmus slúžiaci na klasifikáciu položiek alebo používateľov na základe ich blízkosti v metrickom priestore (geografickom alebo vkusovom).
3. **Clustering (Zhlukovanie):** Automatické rozdeľovanie dát do skupín so spoločnými črtami. Využíva sa najmä pri segmentácii používateľov alebo kategorizácii lokalít.
4. **Matrix Factorization:** Pokročilá technika rozkladu matíc, ktorá transformuje interakcie používateľov do priestoru latentných faktorov, čím rieši problém riedkosti dát (Sparsity).
5. **Neural Networks:** Modely hlbokého učenia schopné modelovať nelineárne vzťahy a spracovávať komplexné dáta zo sociálnych sietí či senzorov mobilných zariadení.

1.8.1 Dizajn a používateľská skúsenosť (UX/UI)

Pri vývoji mobilnej aplikácie nie je kľúčová len funkčná stránka (backend), ale aj spôsob, akým používateľ s aplikáciou interaguje. Návrh používateľského rozhrania sa riadi dvoma základnými disciplínami:

- **UX (User Experience):** Zameriava sa na celkový pocit používateľa pri používaní aplikácie. Cieľom je, aby bola aplikácia intuitívna, logicky usporiadaná a riešila potreby používateľa s minimálnym úsilím.
- **UI (User Interface):** Rieši vizuálnu stránku aplikácie – farby, typografiu, ikony a rozloženie prvkov na obrazovke.

1.9 Architektonické vzory a princípy návrhu

Úspešná mobilná aplikácia musí byť postavená na robustnej architektúre, ktorá zaručuje jej škálovateľnosť a udržateľnosť v ekosystéme rôznych zariadení (telefóny, tablety, ChromeOS či skladacie zariadenia). Podľa oficiálnej dokumentácie spoločnosti Google je základným cieľom architektúry definovať hranice medzi jednotlivými časťami aplikácie a jasne určiť ich zodpovednosť.

1.10 Základné princípy moderného návrhu

Pred definovaním konkrétnych vzorov je potrebné určiť pravidlá, ktoré musí moderná architektúra spĺňať pre zachovanie stability v prostredí s obmedzenými systémovými prostriedkami:

- **Separation of concerns (Oddelenie zodpovedností):** Kľúčový princíp, ktorý zakazuje sústredenie kódu v triedach *Activity* alebo *Fragment*. Tieto komponenty slúžia primárne ako hostitelia používateľského rozhrania, sú krátkodobé (*ephemeral*) a operačný systém ich môže kedykoľvek zničiť z dôvodu nedostatku pamäte alebo zmeny konfigurácie.
- **Drive UI from data models (Riadenie UI dátovými modelmi):** Používateľské rozhranie by malo byť postavené na dátových modeloch, ktoré sú nezávislé od životného cyklu UI prvkov. Perzistentné modely zabezpečujú, že aplikácia nestratí dáta pri reštarte procesu a zostáva funkčná aj pri nestabilnom pripojení k sieti.
- **Single Source of Truth (SSOT - Jednotný zdroj pravdy):** Každý dátový typ má priradeného svojho vlastníka, ktorý ako jediný môže dáta modifikovať. Zmeny sa následne propagujú prostredníctvom imutabilných typov, čo znižuje výskyt chýb a uľahčuje ladenie systému.
- **Unidirectional Data Flow (UDF - Jednosmerný tok dát):** Stav aplikácie prúdi smerom nadol (od SSOT k UI elementom) a udalosti/akcie prúdia smerom nahor. Tento vzor zaručuje konzistenciu dát v celom systéme.

1.11 Porovnanie architektonických vzorov

V histórii vývoja platformy Android sa vyvinulo niekoľko prístupov k organizácii kódu, ktoré reagovali na narastajúcu komplexnosť aplikácií:

1. **MVC (Model-View-Controller):** Tradičný model, kde Controller riadi tok dát. V Androide tento vzor zlyhával, pretože triedy *Activity* preberali úlohu controllera aj view súčasne, čo viedlo k vzniku ťažko udržiavateľného kódu.
2. **MVP (Model-View-Presenter):** Oddelil zodpovednosť medzi View a Presenter. Problémom zostala vysoká previazanosť a nutnosť manuálneho ošetrovania životného cyklu, čo spôsobovalo úniky pamäte (*memory leaks*).
3. **MVVM (Model-View-ViewModel):** Súčasný štandard. *ViewModel* uchováva stav aplikácie nezávisle od UI komponentov, čím rieši problém s reštartovaním aktivít a stratu dát pri zmene orientácie zariadenia.

2 Súčasný stav riešenej problematiky

Na trhu existuje množstvo platforiem zameraných na organizáciu a objavovanie podujatí. Dominantným hráčom v tejto oblasti je služba **Facebook Events**, vyvíjaná spoločnosťou Meta. Vďaka zázemiu veľkej technologickej spoločnosti ide o komplexný systém, ktorý pokrýva široké spektrum potrieb používateľov.

2.1 Facebook Events

Facebook Events umožňuje používateľom vytvárať vyhradené stránky pre konkrétne príležitosti a zasielať verejné alebo súkromné pozvánky. Pre online firmy predstavujú tieto podujatia efektívny nástroj na propagáciu predajov, osláv alebo dôležitých míľnikov. Systém zahŕňa históriu podujatí, možnosť označovania obľúbených aktivít a pokročilé filtrovanie.

- **Výhody:** Obrovská používateľská základňa, vysoká miera integrácie so sociálnou sieťou, prepracované algoritmy na predikciu záujmov a množstvo funkcií dostupných bez priamych poplatkov za vytvorenie podujatia.
- **Nevýhody a obmedzenia:**
  - **Informačné zahltenie a notifikačná únava:** Rozhranie je preplnené reklamami. Nadmerné množstvo systémových upozornení (*Notification Fatigue*) často vedie k tomu, že používatelia dôležité pozvánky na podujatia prehliadnu alebo ignorujú.
  - **Nízka miera kontroly nad dosahom:** Organizátor nemá záruku, že sa informácia o podujatí dostane ku všetkým sledovateľom. Algoritmus uprednostňuje platenú reklamu a interaktívny obsah, čo znevýhodňuje menšie lokálne a komunitné aktivity.
  - **Uzavretosť ekosystému:** Plné využitie funkcií vyžaduje existenciu Facebook profilu, čo odrádza časť modernej populácie (najmä generáciu Z).
  - **Obmedzená vizualizácia:** Mapa podujatí nie je primárnym ovládacím prvkom a je dostupná až po manuálnom rozkliknutí, čo znižuje intuitívnosť pri hľadaní aktivít v okolí.
  - **Absencia priamej komunikácie:** Diskusia v rámci podujatia prebieha formou príspevkov na nástenke, chýba však funkcia reálneho skupinového chatu pre účastníkov v reálnom čase.

Zdroje:
`https://www.facebook.com/events/`
`https://hub.theeventplannerexpo.com/social-media-marketing/the-pros-and-cons-of-facebook-events-should-you-even-bother` https://hub.theeventplannerexpo.com/social-media-marketing/the-pros-and-cons-of-facebook-events-should-you-even-bother
Zdroje:
`https://www.facebook.com/events/`
`https://www.bigcommerce.com/glossary/what-are-facebook-events/`

2.2 Meetup

Platforma **Meetup** je celosvetovo známa platforma sa primárne zameriava na budovanie dlhodobých komunít a záujmových skupín (*Groups*). Na rozdiel od Facebooku, kde sú podujatia často jednorazové, Meetup motivuje používateľov k pravidelnému stretávaniu sa na základe spoločných koníčkov, ako sú technológie, šport či jazykové vzdelávanie.

V porovnaní s Facebook Events ponúka Meetup detailnejšie komunitné funkcie, avšak väčšina z nich je podmienená plateným členstvom:

- **Rozšírená interakcia:** Platforma umožňuje prehliadať zoznamy účastníkov, pristupovať k ich profilom a využívať pokročilé systémy komentárov a správ, čo uľahčuje nadväzovanie kontaktov ešte pred samotným podujatím.
- **Personalizácia (*For You*):** Algoritmus navrhuje podujatia a skupiny na základe histórie aktivít a preferencií používateľa, pričom prioritizuje lokálne komunity.
- **Platený model (Monetizácia):** Najväčšou nevýhodou platformy je, že kľúčové funkcie pre organizátorov (vytváranie podujatí, správa členov) aj pokročilé funkcie pre účastníkov sú spoplatnené. V rámci prémiového balíka získavajú používatelia:
  - prístup k úplným zoznamom a profilom účastníkov,
  - prioritný prístup k podujatiam a čakacím listinám (*Waitlists*),
  - neobmedzené zasielanie správ ostatným členom,
  - rozhranie bez reklám a včasné upozornenia na nové udalosti.
- **Problém s účasťou:** Podobne ako pri iných bezplatných registráciách, aj Meetup čelí vysokej miere tzv. *no-shows* (používatelia, ktorí potvrdia účasť, ale neprirodzene absentujú), čo sťažuje plánovanie kapacity pre organizátorov.

Kritické zhodnotenie: Hoci Meetup poskytuje kvalitnejšie nástroje na sieťovanie (*networking*) než Facebook, jeho uzavretosť za platobnú bránu a slabšia SEO viditeľnosť pre nečlenov otvárajú priestor pre vývoj prístupnejších, bezplatných alternatív.

- **Problém „No-shows“ (Neprihlásení účastníci):** Štatistiky z praxe ukazujú, že v priemere až 50 % používateľov, ktorí potvrdia účasť (*RSVP*), na udalosť nakoniec nepríde. Tento jav je spôsobený nízkou bariérou záväzku, najmä pri bezplatných podujatiach.

- **Vysoké prevádzkové náklady:** Pre organizátorov predstavuje Meetup finančnú záťaž (poplatky od cca 25 $ mesačne), pričom základné balíky výrazne obmedzujú možnosti brandingu, vkladanie multimédií (video, vlastné logá) či prispôsobenie dizajnu skupiny.

- **Geografické obmedzenia vyhľadávania:** Systém je primárne postavený na vyhľadávaní podľa PSČ alebo mesta. To drasticky limituje *SEO* dosah udalostí, ktoré sú organizované online alebo cília na globálne publikum, nakoľko používatelia nevidia relevantné výsledky mimo svojho bezprostredného okolia.

- **Pasivita a „Ghost groups“:** Na platforme sa nachádza veľké množstvo neaktívnych skupín, ktoré slúžia len na zber e-mailových adries (*lead generation*) bez reálnej snahy organizovať stretnutia.

- **Kvalita a bezpečnosť lokalít:** Mnohé stretnutia sa odohrávajú v baroch alebo reštauráciách, čo prináša problémy s hlukom, nevhodným osvetlením alebo dokonca neprofesionálnym správaním účastníkov (alkohol). Verejné inštitúcie (knižnice) sú často pre neformálne skupiny nedostupné.

- **Chýbajúca automatizácia časových pásiem:** Meetup nedostatočne rieši automatický prepočet časových pásiem pri medzinárodných online podujatiach, čo núti organizátorov k manuálnym opravám.

Zhrnutie pre návrh novej aplikácie:
Tieto zistenia naznačujú, že moderná alternatíva by sa mala zamerať na: 1. **Zvýšenie záväznosti účastníkov** (napr. cez integrovaný chat alebo potvrdzovací systém krátko pred akciou). 2. **Transparentnú mapu bez hraníc miest**, ktorá umožní objavovať udalosti organicky scrollovaním. 3. **Natívnu podporu pre online podujatia** s automatickou prácou s časom a SEO optimalizáciou.

2.3 AllEvents.in

AllEvents sa prezentuje ako globálna platforma na objavovanie udalostí, ktorá pokrýva viac než 40 000 miest po celom svete. Platforma sa zameriava na agregáciu širokého spektra podujatí – od komunitných stretnutí až po veľké komerčné koncerty a festivaly.

- **Deklarované funkcie:** Personalizované odporúčania na základe záujmov, sledovanie aktivít priateľov, bezpapierová registrácia (*check-in*) a priama rezervácia vstupeniek v rámci aplikácie.
- **Kategorizácia a distribúcia:** Platforma vyniká prehľadným členením do kategórií, možnosťou exportu do kalendára a funkciou priameho kontaktovania organizátora (*Message to host*).
- **Identifikované nedostatky a technické bariéry:**
  - **Kritická nestabilita aplikácie:** Pri testovaní mobilnej aplikácie boli zaznamenané závažné chyby v procese autentifikácie (*Login failed*) a zlyhania pri verifikácii používateľa cez mobilné číslo, čo znemožňuje plnohodnotné využívanie služieb.
  - **Lokalizačné problémy:** Napriek deklarovanému globálnemu dosahu systém v niektorých prípadoch nedokáže korektne identifikovať menšie mestá, čo núti používateľa k prehliadaniu v režime hosťa (*Guest mode*).
  - **Absencia sociálnej interakcie:** Hoci platforma umožňuje nákup lístkov a sledovanie udalostí, postráda funkciu reálneho skupinového chatu medzi účastníkmi a transparentné zobrazenie zoznamu prihlásených osôb.
  - **Zameranie obsahu:** Databáza je orientovaná primárne na veľké komerčné a spoločenské akcie, čím opomína drobné, spontánne komunitné aktivity.

Zdroje:
`https://play.google.com/store/apps/details?id=com.amitech.allevents`
`https://allevents.in/`

2.4 Hapsy

Aplikácia *Hapsy* sa profiluje ako agregátor podujatí, ktorého cieľom je zjednotiť ponuku z rôznych platforiem (Facebook, Eventbrite, Meetup, GoOut) do jedného interaktívneho zobrazenia na mape. Jej hlavnou ambíciou je eliminovať potrebu prepínania medzi viacerými aplikáciami.

- **Kľúčové funkcie:** Agregácia dát z externých zdrojov, personalizované upozornenia na základe preferencií používateľa, ukladanie obľúbených udalostí a prehľadná mapa mesta.
- **Identifikované obmedzenia a používateľská kritika:**

- **Geografická limitácia:** Podľa spätnej väzby používateľov trpí aplikácia nedostatkom lokálneho obsahu mimo hlavných miest. Algoritmus sa zameriava na veľké komerčné databázy, čím opomína komunitný život v menších regiónoch.
- **Absencia tvorby obsahu:** Systém funguje primárne ako prehliadač (*viewer*). Bežný používateľ nemá možnosť vytvoriť vlastnú udalosť, čo znemožňuje organizáciu spontánnych aktivít (napr. spomínané amatérske športové zápasy).
- **Technická nedokonalosť:** Používateľské recenzie naznačujú prítomnosť kritických chýb v rozhraní (*UI bugs*), ako sú problémy s posúvaním obsahu (*scrolling*), čo znižuje komfort používania.
- **Obmedzená interakcia:** Podobne ako pri iných agregátoroch, aj tu chýba sociálna vrstva – možnosť priamej komunikácie medzi účastníkmi alebo správa tímov.

Zdroje:
`https://play.google.com/store/apps/details?id=com.hapsy.app`
`https://mapibara.com/`

2.5 Mapibara

Platforma *Mapibara* predstavuje minimalistický prístup k objavovaniu podujatí, pričom kladie absolútny dôraz na mapové rozhranie a anonymitu používateľa. Na rozdiel od komplexných sociálnych sietí sa zbavuje povinných registrácií a zameriava sa na okamžitý prístup k informáciám.

- **Silné stránky dizajnu:** Extrémne jednoduché a prehľadné používateľské rozhranie (*UI*), kde mapa zaberá celú plochu obrazovky. Absencia prihlasovacieho procesu odstraňuje vstupnú bariéru pre nových používateľov.
- **Identifikované kritické nedostatky:**
  - **Absencia obsahu:** Pri praktickom testovaní platforma vykazuje takmer nulovú naplnenosť podujatiami, čo ju robí v súčasnom stave nepoužiteľnou.
  - **Chýbajúca interakcia:** Kvôli absencii používateľských profilov nie je možné s účastníkmi komunikovať, spravovať kapacity ani riešiť logistiku (napr. spomínané rozdelenie do tímov pri športe).
  - **Pasivita systému:** Platforma slúži len ako statický prehliadač bodov na mape bez možnosti aktívneho zapojenia komunity.

Ponaučenie pre vlastný návrh: Analýza Mapibary ukazuje, že samotná vizualizácia na mape nestačí. Pre úspech aplikácie je nevyhnutné skombinovať **jednoduchosť mapy** so **sociálnou interakciou** (prihlásenie, chat), aby používatelia mali dôvod podujatia nielen hľadať, ale aj vytvárať a navštevovať.

2.6 Zhrnutie analýzy súčasného stavu

Na základe podrobnej analýzy dominantných platforiem (Facebook Events, Meetup, AllEvents, Hapsy a Mapibara) je možné definovať súbor štandardných funkcií, ktoré tvoria základ moderných systémov na správu podujatí, ale zároveň identifikovať kritické funkčné medzery.

Štandardné funkcie prítomné v analyzovaných riešeniach: Väčšina spomenutých aplikácií disponuje nasledujúcim funkčným základom:

- **Objavovanie a vizualizácia:** Interaktívna mapa s podujatiami, vyhľadávanie podľa kategórií a pokročilé filtrovanie.
- **Informačná vrstva:** Detailný popis podujatia, fotogaléria, profil organizátora a zdieľanie (*sharing*) na iné platformy.
- **Personalizácia:** História navštívených udalostí, systém odporúčaných podujatí (*Recommended for you*) a push notifikácie.
- **Interakcia:** Možnosť vytvorenia vlastnej verejnej alebo súkromnej udalosti po predchádzajúcej registrácii.

Identifikácia funkčných medzier (Inovačný potenciál): Napriek širokej škále funkcií analýza odhalila oblasti, ktoré súčasné riešenia pokrývajú nedostatočne, čo vytvára priestor pre vývoj nového, špecializovaného riešenia:

- **Dynamický manažment kapacity:** Absencia automatizovaného systému náhradníkov (*Waiting List*). Väčšina platforiem neponúka riešenie pre prípady, kedy je kapacita naplnená a nie je možné automaticky osloviť ďalšieho záujemcu v poradí pri uvoľnení miesta.
- **Verifikácia reálnej účasti:** Kritický nedostatok nástrojov na potláčanie *no-show* efektu (používatelia, ktorí potvrdia účasť, ale neprídu). Chýbajú mechanizmy ako **GPS Check-in** alebo verifikácia prítomnosti na mieste činu.
- **Komunitná logistika:** Absentujú nástroje na správu spoločného vybavenia (*Inventory sharing* – napr. kto prinesie športové náčinie) a funkcia reálneho skupinového chatu pre všetkých účastníkov v reálnom čase.

- **Moderovanie a bezpečnosť:** Pri raste používateľmi generovaného obsahu chýbajú v mnohých aplikáciách citlivé nástroje na nahlasovanie (*reporting*) a včasné blokovanie nevhodného obsahu alebo používateľov.

- **Regionálna relevantnosť:** Nízka hustota obsahu v menších mestách a regiónoch mimo hlavných metropol, čo je dôsledkom orientácie veľkých platforiem na komerčné a masové podujatia.

2.7 Syntéza poznatkov a východiská pre vlastný návrh

Na základe vykonanej analýzy existujúcich riešení je možné konštatovať, že hoci trh ponúka robustné platformy, žiadna z nich nepokrýva komplexne potreby malých, spontánnych komunít s dôrazom na logistiku a potláčanie negativity (napr. spomínaný *no-show* efekt).

Pre lepšiu prehľadnosť sú kľúčové rozdiely medzi analyzovanými platformami a navrhovaným riešením zhrnuté v Tabuľke 3.

Tabuľka 3: Porovnanie kľúčových funkcií existujúcich platforiem a navrhovanej aplikácie

Funkcia / Platforma	**FB**	**Meetup**	**AllEvents**	**Hapsy**	**Mapibara**
Mapa ako primárne UI	Nie	Čiastočne	Áno	Áno	Áno
Tvorba udalostí	Áno	Platené	Áno	Nie	Nie
Skupinový chat	Nie	Áno	Nie	Nie	Nie
Manažment náhradníkov	Nie	Áno	Nie	Nie	Nie
Ochrana „No-show“	Nie	Nie	Nie	Nie	Nie
Zameranie na komunity	Áno	Áno	Nie	Nie	Nie

Definovanie smerovania vlastného riešenia Zistené nedostatky konkurenčných riešení priamo formujú funkčné požiadavky na moju aplikáciu. Vlastné riešenie sa bude odlišovať predovšetkým v troch pilieroch:

1. **Interaktivita namiesto pasivity:** Integrácia komunikačného modulu (chat) priamo do detailu udalosti, čo umožní operatívne riešiť zmeny (napr. počasie, meškanie) v reálnom čase.

2. **Zvýšenie kredibility účastníkov:** Implementácia overovacích mechanizmov (napr. potvrdzovanie prítomnosti na mieste cez polohu zariadenia), čo motivuje používateľov k reálnej účasti.

3. **Minimalizmus a dostupnosť:** Odstránenie finančných bariér pre organizátorov a zachovanie čistého mapového rozhrania pre okamžitý prehľad o dianí v okolí bez zbytočného informačného balastu.

Tento prístup umožní vytvoriť nástroj, ktorý nie je len „nástenkou podujatí“, ale živým ekosystémom pre lokálne komunity.
Zhrnutím možno konštatovať, že existuje dopyt po riešení, ktoré by premostilo priepasť medzi statickou informáciou o udalosti a aktívnou logistickou podporou komunitných aktivít.

3 Použité technológie

3.1 Vývojové prostredie a platforma

3.1.1 Operačný systém Android

Táto práca je zameraná na vývoj mobilnej aplikácie pre operačný systém **Android**. Dôvodom zvolenia tejto platformy je jej dominantné postavenie na trhu mobilných zariadení, kde podľa aktuálnych štatistík zaujíma viac ako 70 % podiel. Android ponúka široké možnosti prispôsobenia a hlbokej integrácie s hardvérovými službami (ako sú lokalizačné služby), čo je kľúčové pre praktickú časť tejto práce.
(Zdroj: `https://gs.statcounter.com/android-version-market-share`)

3.1.2 Distribúcia a ekosystém platforiem

Voľba operačného systému ovplyvňuje nielen technologický zásobník, ale aj stratégiu uvedenia na trh. Platforma Android je často preferovaná pre nasadenie minimálneho životaschopného produktu (*MVP*) kvôli flexibilnejším pravidlám v obchode Google Play v porovnaní s uzavretým ekosystémom iOS.

Hoci existujú multiplatformové riešenia, pre účely tejto práce bol zvolený natívny prístup z dôvodu maximálnej optimalizácie výkonu a prístupu k systémovým API. Porovnanie prístupov na základe analýzy [**pullur_medium_2025**] uvádza Tabuľka 4.

Tabuľka 4: Súhrnné porovnanie natívneho a multiplatformového vývoja

Metrika	**Native (Kotlin)**	**Flutter**	**React Native**	**KMM**
Veľkosť aplikácie	**Najmenšia**	Väčšia	Väčšia	Stredná
Rýchlosť štartu	**Najrýchlejšia**	Stredná	Pomalšia	Rýchla
Animácie	**Plynulé**	Plynulé	Riziko lagov	–
Zdieľanie kódu	Žiadne	95 % +	90 % +	Logika
Spotreba batérie	**Nízka**	Stredná	Vysoká	Nízka

3.2 Programovací jazyk Kotlin

Kotlin je moderný, staticky typovaný jazyk, ktorý je od roku 2019 preferovanou voľbou spoločnosti Google pre vývoj aplikácií na platformu Android. Medzi jeho kľúčové prínosy pre túto prácu patria:

- **Null Safety:** Výrazne znižuje riziko pádov aplikácie (chyba *NullPointerException*) vďaka striktnému rozlišovaniu nulovateľných typov už v čase kompilácie.
- **Coroutines:** Umožňujú efektívne vykonávanie asynchrónnych úloh (napr. sťahovanie dát o podujatiach z cloudu) bez blokovania hlavného vlákna a zamŕzania používateľského rozhrania.
- **Stručnosť:** Redukcia nadbytočného kódu (*boilerplate*) umožňuje rýchlejšiu implementáciu biznis logiky odporúčacieho algoritmu.

3.3 UI Toolkit: Jetpack Compose

Na tvorbu používateľského rozhrania bol zvolený moderný deklaratívny toolkit **Jetpack Compose**. Na rozdiel od staršieho systému založeného na XML súboroch, Compose umožňuje definovať UI priamo v jazyku Kotlin.

- **Deklaratívny prístup:** Programátor opisuje výsledný stav rozhrania, pričom framework sa automaticky stará o jeho prekreslenie pri zmene dát (napr. pri aktualizácii zoznamu podujatí).
- **Komponentový dizajn:** UI je vyskladané z malých, znovupoužiteľných funkcií (*Composables*), čo uľahčuje údržbu a testovanie aplikácie.

3.4 Vývojové nástroje

Ako už sám naznačuje názov, Android Studio je oficiálne vývojové prostredie (IDE) bohaté na funkcie pre vývoj aplikácií pre Android, ktoré integruje nástroje na návrh, kódovanie, tvorbu a testovanie. Ponúka natívnu podporu pre Kotlin, Java a C++, rozsiahlu rozšíriteľnosť pluginov a bezproblémovú emuláciu naprieč typmi zariadení. Pravidelné aktualizácie zabezpečujú kompatibilitu s novými verziami systému Android, pokročilé ladenie a analýzu výkonu v jednotnom prostredí a preto je ideálnou voľbou pre tento projekt.

3.5 Technická špecifikácia a použité knižnice

3.6 Android SDK a správa verzií

Kľúčovým pilierom vývoja je **Android SDK** (*Software Development Kit*), čo je sada vývojových nástrojov poskytovaná spoločnosťou Google. SDK obsahuje knižnice, ladiace nástroje a emulátory potrebné na transformáciu zdrojového kódu do spustiteľného balíka aplikácie.

V rámci projektu je správa verzií riešená definovaním úrovní rozhraní API (*API Levels*), ktoré určujú rozsah kompatibility a dostupné systémové funkcie [**geeks_android_sdk**].

- **minSdk 26 (Android 8.0 Oreo):** Táto hodnota stanovuje najnižšiu možnú verziu systému, na ktorej je možné aplikáciu spustiť. Voľba verzie 26 bola zvolená ako optimálny kompromis medzi dostupnosťou moderných API (napr. vylepšené kanály notifikácií či správa úloh na pozadí) a širokým pokrytím trhu, ktoré aktuálne presahuje 90 % aktívnych zariadení.

- **targetSdk 35 (Android 15):** Informuje operačný systém, pre ktorú najnovšiu verziu bola aplikácia optimalizovaná a testovaná. Použitie aktuálneho SDK zabezpečuje, že aplikácia dodržiava najnovšie bezpečnostné štandardy a protokoly ochrany súkromia, ktoré systém Android vyžaduje pre moderné aplikácie v obchode Google Play.

Rozdiel medzi týmito úrovňami definuje spätnú kompatibilitu. Zatiaľ čo *minSdk* garantuje spustenie aplikácie na staršom hardvéri, *targetSdk* umožňuje systému aktivovať špecifické optimalizácie výkonu a zabezpečenia určené pre najnovšie generácie mobilných zariadení. https://www.geeksforgeeks.org/android/android-sdk-and-its-components/

3.6.1 Dizajnový systém Material Design 3

Vizuálna identita a používateľské rozhranie aplikácie sú postavené na najnovšom štandarde **Material Design 3** (známy aj ako *Material You*). Ide o adaptívny systém usmernení, komponentov a nástrojov vyvinutý spoločnosťou Google, ktorý podporuje osvedčené postupy pri navrhovaní moderných používateľských rozhraní [**material_io_m3**].

Hlavným prínosom integrácie Material Design 3 do projektu je zefektívnenie spolupráce medzi dizajnom a vývojom, čo umožňuje rýchle budovanie estetických a funkčných produktov.

- **Komponentové sady:** Material 3 poskytuje širokú škálu sád komponentov, ktoré sú špecificky navrhnuté pre rôzne prípady použitia a veľkosti obrazoviek. V projekte sú využité kľúčové prvky ako `TopAppBar` (horná lišta aplikácie), `NavigationDrawer` (bočné navigačné menu) a `Card` (karty pre prehľadné zobrazenie informácií o podujatiach).

- **Adaptivita a prístupnosť:** Systém je navrhnutý tak, aby sa automaticky prispôsoboval rôznym rozlíšeniam a orientáciám displeja, pričom kladie veľký dôraz na čitateľnosť a prístupnosť (*accessibility*) pre všetkých používateľov.

- **Integrácia s Jetpack Compose:** Využitie knižnice `androidx.compose.material3` umožňuje natívne deklaratívne definovanie štýlov, farieb a typografie priamo v kóde Kotlin, čím sa zabezpečuje vizuálna konzistencia naprieč celou aplikáciou.

3.7 Externé softvérové knižnice

Pre rozšírenie základných funkcií systému Android a zabezpečenie plynulého používateľského zážitku boli do projektu integrované špecializované knižnice. Tieto nástroje riešia kritické úlohy od navigácie v deklaratívnom prostredí až po nízkoúrovňovú komunikáciu s hardvérom kamery.

3.7.1 Navigation Compose (2.8.5)

Knižnica **Navigation Compose** poskytuje infraštruktúru pre riadenie toku obrazoviek v aplikáciách postavených na Jetpack Compose. Umožňuje definovať navigačný graf priamo v kóde Kotlin, čím nahrádza staršie riešenia založené na XML súboroch a fragmentoch.

- **NavHost:** Slúži ako vizuálny kontajner, ktorý dynamicky vymieňa jednotlivé kompozitné funkcie (*composables*) na základe aktuálnej trasy.
- **NavController:** Centrálny objekt, ktorý spravuje zásobník obrazoviek (*backstack*) a zabezpečuje správnu reakciu na systémové tlačidlo „Späť“.
- **Typová bezpečnosť:** Verzia 2.8.5 prináša natívnu podporu pre typovo bezpečnú navigáciu, čo eliminuje chyby pri prenose argumentov (napr. ID podujatia) medzi obrazovkami.

3.7.2 CameraX (1.3.4)

CameraX je moderná knižnica zameraná na prácu s fotoaparátom, ktorá odstraňuje zložitosť spojenú so správou špecifík hardvéru od rôznych výrobcov. Je postavená na koncepte prípadov použitia (*use-cases*), z ktorých boli v projekte implementované najmä:

- **Preview:** Zabezpečuje plynulý živý náhľad obrazu z kamery priamo v používateľskom rozhraní.
- **Image Analysis:** Poskytuje prístup k obrazovým dátam v reálnom čase, čo je kľúčové pre **algoritmy na skenovanie a dekódovanie QR kódov** pri vstupe na podujatia.

- **Image Capture:** Umožňuje zachytávanie fotografií (napr. pre profilové fotky alebo dokumentáciu podujatia) s automatickou optimalizáciou expozície a zaostrenia.

3.7.3 Coil (2.7.0)

Názov **Coil** predstavuje akronym pre *Coroutine Image Loader*. Ide o efektívnu knižnicu na asynchrónne načítavanie obrázkov z cloudu (Firebase Storage), ktorá je plne optimalizovaná pre prostredie Kotlin Coroutines.

- **Optimalizácia výkonu:** Knižnica automaticky vykonáva operácie ako cachovanie na disk a do pamäte RAM, zmenu mierky obrázkov (*downsampling*) a inteligentné rušenie sieťových požiadaviek pri odscrollovaní prvku z obrazovky.

- **Integrácia s Compose:** Vďaka natívnej podpore pre Jetpack Compose umožňuje zobrazenie obrázkov s minimálnym množstvom riadiaceho kódu (*boilerplate*), čím zvyšuje čitateľnosť projektu.

3.8 Architektúra projektu

V projekte využívame architektúru **MVVM** (*Model-View-ViewModel*) v kombinácii s **Repository pattern**. Tento prístup je oficiálne odporúčaný spoločnosťou Google a predstavuje súčasný štandard v Android komunite pre budovanie škálovateľných a udržiavateľných aplikácií.

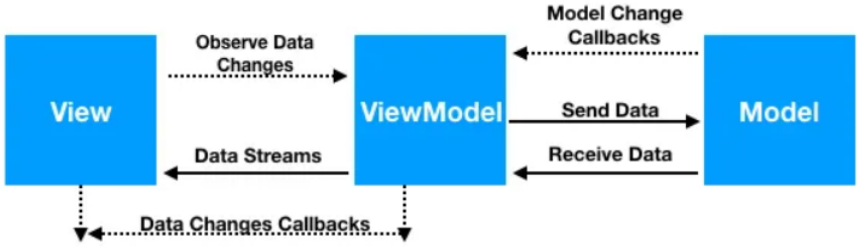
Model MVVM pozostáva z troch základných komponentov, z ktorých každý má jasne definovanú zodpovednosť:

- **Model:** Reprezentuje dátovú vrstvu a biznis logiku. V našom prípade sem patrí komunikácia s Firebase a správa lokálnych dát.

- **View (Zobrazenie):** Používateľské rozhranie, ktoré pasívne zobrazuje dáta a odosiela akcie používateľa do ViewModelu.

- **ViewModel (Model zobrazenia):** Pôsobí ako prostredník, ktorý pripravuje dáta z modelu pre zobrazenie a udržiava stav aplikácie nezávisle od životného cyklu rozhrania.

Hlavným dôvodom voľby MVVM sú jeho prínosy pre stabilitu projektu, najmä **zvýšená testovateľnosť** (možnosť testovať logiku bez UI), **jednoduchšia údržba** vďaka oddeleniu zodpovedností a **odolnosť voči zmenám konfigurácie** (napr. otočenie displeja), kedy ViewModel chráni dáta pred stratou.



Obr. 1: Diagram architektúry MVVM s Repository pattern v systéme Android

3.8.1 Moderný prístup: MVVM v kombinácii s Jetpack Compose

V súlade s princípmi čistého návrhu (*Clean Architecture*) opísanými v [**sks_medium_2024**], je v projekte implementovaný vzor MVVM s dôrazom na **jednosmerný tok dát** (*Unidirectional Data Flow - UDF*). Tradičný mechanizmus *Data Bindingu* známy zo starších verzií Androidu je tu nahradený moderným reaktívnym prístupom:

- **State Hoisting (Vynášanie stavu):** ViewModel uchováva kompletný stav obrazovky v podobe imutabilného objektu (napr. *EventsUiState*). UI komponenty sú takmer bezstavové (*stateless*), čo uľahčuje ich znovupoužiteľnosť.
- **Reaktívne pozorovanie (StateFlow):** Používateľské rozhranie je prepojené s *StateFlow* vo ViewModele. Akákoľvek zmena v dátach (napr. aktualizácia zoznamu podujatí) vyvolá okamžitú **rekompozíciu** (automatické prekreslenie) dotknutých častí rozhrania.
- **Spracovanie udalostí (Events):** Všetky akcie používateľa sú prenášané ako udalosti smerom k ViewModelu. View nikdy neupravuje dáta priamo, čo zaručuje, že *ViewModel* zostáva jediným zdrojom pravdy (*Single Source of Truth*).

3.9 Platforma Firebase

Firebase predstavuje komplexnú platformu cloudových služieb od spoločnosti Google, ktorá je navrhnutá na zrýchlenie vývoja, zvýšenie bezpečnosti a zabezpečenie škálovateľnosti moderných aplikácií. Platforma pokrýva celý životný cyklus vývoja softvéru a je využívaná svetovými lídrami ako Duolingo, NPR či Venmo.

3.9.1 Dôvody voľby a kľúčové výhody

Voľba tejto platformy pre účely mojej práce vychádza z jej schopnosti riešiť zložité vývojové výzvy pomocou integrovaných produktov:

- **Efektívna autentifikácia:** Poskytuje robustné knižnice pre správu používateľov, overovanie hesiel a integráciu poskytovateľov tretích strán (Google, GitHub atď.).
- **Škálovateľné úložisko:** Cloud Storage umožňuje efektívnu prácu s používateľským obsahom, ako sú obrázky a videá, s vysokou úrovňou zabezpečenia.
- **Real-time synchronizácia:** NoSQL databázy (Firestore, Realtime Database) umožňujú synchronizáciu dát naprieč klientmi v reálnom čase, a to aj v offline režime.
- **Bezpečnosť a správa:** Flexibilné bezpečnostné pravidlá chránia dáta pred neoprávneným prístupom priamo na úrovni cloudu.
- **Serverless logika:** Cloud Functions umožňujú rozšíriť funkcionalitu aplikácie o asynchrónne spracovanie na pozadí bez nutnosti správy vlastného servera.

3.10 Firebase Authentication

Firebase Authentication poskytuje backendové služby, ľahko použiteľné súpravy SDK a hotové knižnice používateľského rozhrania na autentifikáciu používateľov v aplikácii. Podporuje overovanie pomocou hesiel, telefónnych čísiel a populárnych poskytovateľov identít (Google, Facebook, Twitter). Služba využíva moderné priemyselné štandardy ako *OAuth 2.0* a *OpenID Connect*, pričom zabezpečuje kompletnú správu relácií a overovanie e-mailových adries.

3.11 Firebase Firestore

Cloud Firestore je flexibilná, škálovateľná NoSQL databáza určená pre mobilný a webový vývoj. Udržiava dáta synchronizované medzi klientskymi aplikáciami prostredníctvom reaktívnych poslucháčov (*realtime listeners*).

Kľúčovou vlastnosťou pre túto prácu je **offline podpora**, ktorá umožňuje aplikácii plynule fungovať aj pri nestabilnom internetovom pripojení tým, že lokálne cachuje potrebné dáta a synchronizuje ich hneď po obnovení siete.

3.12 Firebase Storage

Cloud Storage pre Firebase je výkonná a nákladovo efektívna služba určená na ukladanie objektov, ako sú fotografie podujatí či profilové obrázky používateľov. Súpravy SDK zabezpečujú vysokú mieru spoľahlivosti pri nahrávaní a sťahovaní súborov bez ohľadu na kvalitu sieťového pripojenia, pričom prístup k súborom je striktne riadený integračnými pravidlami Firebase Authentication.

3.13 Firebase

Firebase predstavuje súbor nástrojov určených na budovanie, zlepšovanie a rozširovanie aplikácií. Poskytované služby pokrývajú veľkú časť funkcionalít, ktoré by si vývojári za bežných okolností museli programovať sami, hoci sa primárne chcú sústrediť na samotný používateľský zážitok (UX). Tieto služby zahŕňajú analytiku, autentifikáciu, databázy, konfiguráciu, úložiská súborov, push správy a mnohé ďalšie. Všetky služby sú hostované v cloude a vyžadujú si od vývojára minimálne úsilie pri ich škálovaní.

3.14 Firebase Authentication

Firebase Authentication poskytuje backendové služby, ľahko použiteľné súpravy SDK a hotové knižnice používateľského rozhrania na autentifikáciu používateľov v aplikácii. Podporuje overovanie pomocou hesiel, telefónnych čísiel, populárnych poskytovateľov identít (Google, Facebook, Twitter) a ďalšie.

Integruje sa s ostatnými službami Firebase a využíva štandardy ako *OAuth 2.0* a *OpenID Connect*.

3.15 Firebase Firestore

Cloud Firestore je flexibilná, škálovateľná databáza pre vývoj mobilných, webových a serverových aplikácií. Udržiava dáta synchronizované medzi klientskymi aplikáciami prostredníctvom *realtime listeners* a ponúka offline podporu pre mobilné zariadenia a web, vďaka čomu aplikácie fungujú bez ohľadu na sieťovú latenciu.

3.16 Firebase Storage

Cloud Storage pre Firebase je výkonná a nákladovo efektívna služba na ukladanie objektov (súborov). Súpravy Firebase SDK pridávajú k nahrávaniu a sťahovaniu súborov vysokú úroveň zabezpečenia bez ohľadu na kvalitu sieťového pripojenia.

3.17 Definícia a význam odporúčacích systémov

S prudkým rozvojom internetových technológií a inteligentných mobilných zariadení čelí moderný používateľ fenoménu informačného preťaženia. V prostredí s takmer neobmedzeným množstvom digitálneho obsahu a služieb sa odporúčacie systémy (angl. *Recommender Systems*) stali kľúčovým nástrojom na personalizovanú filtráciu informácií.

Ich primárnou úlohou je predpovedať preferencie používateľa a zredukovať rozsiahly priestor dostupných položiek na úzky výber relevantných možností, čím sa zvyšuje celková spokojnosť a efektivita interakcie s platformou. Súčasný výskum sa čoraz viac

orientuje na spracovanie implicitných dát. Ide o behaviorálne vzorce, ako sú kliknutia (*click-stream*), história navštívených miest či čas strávený interakciou s konkrétnym objektom [**electronics11010141**].

3.17.1 Obsahové filtrovanie (Content-Based Filtering)

Tento prístup je založený na analýze špecifických atribútov položiek. Systém identifikuje vlastnosti objektov (napr. textové informácie, kategórie alebo tagy), o ktoré používateľ v minulosti prejavil záujem, a na ich základe generuje odporúčania pre podobné položky [**electronics11010141**].

- **Princíp:** Analýza atribútov a metadát položiek (napr. žáner, kategória) s cieľom nájsť podobnosť s preferenciami používateľa.
- **Využitie:** Efektívne najmä pri službách pracujúcich s textovými dátami, hudbou alebo e-commerce [**electronics11010141**].
- **Limity:** Nízka miera diverzity (angl. *serendipity*) – používateľovi nie sú ponúkané nové, neobjavené kategórie obsahu, čo vedie k efektu informačnej bubliny.

3.17.2 Kolaboratívne filtrovanie (Collaborative Filtering)

Kolaboratívne filtrovanie predpovedá vkus používateľa na základe analýzy správania a hodnotení celej komunity používateľov (tzv. „wisdom of the crowd“).

- **Memory-based:** Hľadá podobnosť priamo medzi používateľmi (*User-based*) alebo položkami (*Item-based*) pomocou algoritmov ako KNN alebo korelačných koeficientov.
- **Model-based:** Využíva pokročilé štatistické metódy a techniky strojového učenia, ako sú zhlukovanie (*Clustering*), *Singular Value Decomposition* (SVD) alebo *Principal Component Analysis* (PCA).
- **Hlavné nedostatky:** Zdroj [**electronics11010141**] definuje tri kritické obmedzenia:
  - *Cold Start:* Problém s odporúčaním položiek pre nových používateľov bez histórie interakcií.
  - *Sparsity (Riedkosť dát):* Situácia, keď je v systéme príliš málo hodnotení na to, aby bolo možné spoľahlivo vypočítať podobnosť.
  - *Gray Sheep (Sivé ovce):* Používatelia s veľmi unikátnym vkusom, ktorí nevykazujú podobnosť so žiadnou väčšou skupinou používateľov, čo sťažuje generovanie odporúčaní.

3.17.3 Hybridné systémy

Hybridné modely kombinujú viaceré techniky s cieľom eliminovať slabiny jednotlivých prístupov, najmä problém riedkosti dát (*sparsity*) a studeného štartu. Podľa Burkeho klasifikácie [**electronics11010141**] existuje sedem základných metód hybridizácie, ktoré sú uvedené v tabuľke 5. Hlavným cieľom väčšiny štúdií zaoberajúcich sa hybridnými

Tabuľka 5: Typy hybridných odporúčacích systémov podľa Burkeho [**electronics11010141**]

Metóda	**Princíp fungovania**
Weighted	Skóre z rôznych techník sa kombinuje pomocou numerických váh.
Switching	Systém prepína medzi technikami v závislosti od kontextu.
Mixed	Odporúčania z viacerých zdrojov sú zobrazené súčasne.
Feature Combination	Atribúty z jedného zdroja slúžia ako vstupné črty pre druhý model.
Feature Augmentation	Výstup jednej techniky sa stáva parametrom pre ďalšie spracovanie.
Cascade	Postupné zjemňovanie poradia (jeden systém vytvorí rebríček, druhý ho upresní).
Meta-level	Celý model vygenerovaný jednou technikou slúži ako vstup pre druhú.

modelmi po roku 2010 je integrácia tzv. vedľajších informácií (*side information*), ako sú sociálne väzby medzi používateľmi alebo kontextové dáta, ktoré dopĺňajú chýbajúce explicitné hodnotenia. Týmto spôsobom hybridné systémy dosahujú vyššiu presnosť a robustnosť v porovnaní so samostatnými modelmi.

3.18 Techniky spracovania dát v odporúčacích systémoch

Pre efektívne fungovanie vyššie spomínaných modelov sa využívajú špecifické techniky dolovania dát (Data Mining):

1. **Text Mining:** Analýza neštruktúrovaných textov pomocou sémantickej analýzy a váhovania pojmov (napr. TF-IDF). Slúži na pochopenie obsahu podujatí a preferencií používateľa.

2. **K-Nearest Neighbor (KNN):** Algoritmus slúžiaci na klasifikáciu položiek alebo používateľov na základe ich blízkosti v metrickom priestore (geografickom alebo vkusovom).

3. **Clustering (Zhlukovanie):** Automatické rozdeľovanie dát do skupín so spoločnými črtami. Využíva sa najmä pri segmentácii používateľov alebo kategorizácii lokalít.

4. **Matrix Factorization:** Pokročilá technika rozkladu matíc, ktorá transformuje interakcie používateľov do priestoru latentných faktorov, čím rieši problém riedkosti dát (Sparsity).

5. **Neural Networks:** Modely hlbokého učenia schopné modelovať nelineárne vzťahy a spracovávať komplexné dáta zo sociálnych sietí či senzorov mobilných zariadení.

3.19 Metriky hodnotenia kvality odporúčaní

Na overenie úspešnosti algoritmov a kvantifikáciu kvality odporúčaní sa využívajú metriky založené na porovnaní predpovedí systému so skutočnými preferenciami používateľa. Základným nástrojom pre tento výpočet je tzv. konfúzna matica (*Confusion Matrix*), ktorá klasifikuje výsledky do štyroch kategórií podľa tabuľky 6 [**electronics11010141**].

Tabuľka 6: Konfúzna matica pre odporúčacie systémy [**electronics11010141**]

	Položka relevantná	**Položka nerelevantná**
Odporúčaná	True Positive (TP)	False Positive (FP)
Neodporúčaná	False Negative (FN)	True Negative (TN)

Na základe týchto hodnôt definujeme nasledujúce kvalitatívne indikátory:

- **Presnosť (Precision):** Pomer relevantných odporúčaných položiek k celkovému počtu odporúčaní. Vyjadruje mieru úspešnosti systému pri výbere položiek.

$$Precision = \frac{TP}{TP + FP}$$

- **Úplnosť (Recall):** Pomer úspešne odporúčaných položiek k celkovému počtu všetkých relevantných položiek, ktoré mal systém nájsť. Recall je identický s metrikou TPR (*True Positive Rate*) [**electronics11010141**].

$$Recall = \frac{TP}{TP + FN}$$

- **F-Measure:** Harmonický priemer presnosti a úplnosti. Táto metrika sa využíva najmä kvôli faktu, že medzi Precision a Recall existuje vzťah nepriamej úmery (*trade-off*).

$$F = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

- **Presnosť (Accuracy):** Pomer všetkých správnych predpovedí (TP + TN) k celkovému počtu prípadov. Zdroj [**electronics11010141**] však upozorňuje, že pri nevyvážených dátach (*unbalanced data*) môže byť táto metrika zavádzajúca.

- **RMSE (Root Mean Squared Error):** Index využívaný na hodnotenie presnosti predikcie číselného hodnotenia. Meria smerodajnú odchýlku rozdielov medzi predpovedaným a skutočným hodnotením.

- **ROC a AUC:** ROC krivka vizualizuje vzťah medzi mierou falošných pozitív (FPR) a mierou skutočných pozitív (TPR). Keďže krivka je grafické vyjadrenie, pre kvantitatívne porovnanie sa využíva plocha pod touto krivkou – **AUC** (*Area Under the Curve*). Hodnota AUC nad 0,8 sa považuje za indikátor vysokej presnosti modelu [**electronics11010141**].

3.20 Google Maps Platform

V rámci geolokačných služieb aplikácia využíva dva kľúčové komponenty platformy Google Maps, ktoré umožňujú vizualizáciu dát a interakciu s konkrétnymi miestami.

3.20.1 Maps SDK for Android

Maps SDK for Android umožňuje pridávať mapové podklady do aplikácie (vrátane Wear OS) s využitím údajov Google Maps, zobrazenia máp a reakcií na gestá. SDK poskytuje nástroje na vkladanie doplňujúcich informácií o polohách a podporuje interakciu používateľa prostredníctvom značiek (*Markers*), polygónov a iných prekrytí (*Overlays*).

Platforma plne podporuje programovacie jazyky Kotlin a Java, pričom poskytuje dodatočné knižnice a rozšírenia pre pokročilé funkcie a moderné programovacie techniky.

3.20.2 Places SDK for Android

Places SDK for Android umožňuje vytvárať aplikácie, ktoré kontextuálne reagujú na okolité podniky a iné miesta v blízkosti zariadenia. Tento nástroj dopĺňa základné geografické služby systému Android o bohatú databázu miest, ktoré majú pre používateľa konkrétny význam.

Kľúčové funkcie a rozhrania zahŕňajú:

- **Programový prístup k databáze:** SDK poskytuje prístup k informáciám o miestnych podnikoch, bodoch záujmu a aktuálnej polohe zariadenia.
- **Autocomplete:** Ponúka predpripravené komponenty (*Widgets*) na predpovedanie názvov miest v reakcii na vyhľadávacie dopyty používateľa.
- **Definícia miesta (*Place*):** Miesto je v rámci API reprezentované rozhraním *Place*. Definujeme ho ako fyzický priestor s názvom (napr. firmy, pamiatky, geografické polohy). Objekt *Place* obsahuje údaje ako názov, adresa, ID miesta, geografická poloha, telefónne číslo, typ miesta či webová adresa.

Podobne ako Maps SDK, aj tento nástroj plne podporuje jazyky Kotlin a Java vrátane rozšírení pre pokročilý vývoj.

4 Návrh riešenia

V tejto sekcii sa budeme venovať samotnému návrhu riešenia našej aplikácie začínajúc od funkčných a nefunkčných požiadaviek, cez grafické návrhy a diagrami až po samotnú implemntáciu a testovanie

4.0.1 Funkcionálne a Nefunkcionálne požiadavky

Definivanie funkcionálnych a nefunkcionálnych požiadaviek je podstatným krokom návrhu riešenia, pretože nám pomáha jasne stanoviť správanie aplikácie ktoré by splnili jej cieľ

4.0.2 Funkcionálne požiadavky

Funkcionálne požiadavky definujú správanie systému a služby, ktoré používateľ od systému očakáva. Ich hlavným cieľom je:

1. Definovať funkcie, ktoré sa od systému očakávajú.
2. Napomôcť k lepšiemu pochopeniu funkcionality systému.
3. Identifikovať, čo systém musí alebo nesmie vykonávať.
4. Umožniť fungovanie systému (funkčnosť nie je priamo závislá od nefunkcionálnych požiadaviek).
5. Zabezpečiť, že systém bude spĺňať reálne potreby používateľov.
6. Definovať prvky nevyhnutné pre samotný chod systému.
7. Stanoviť povinné body, ktoré musia byť v systéme implementované.
8. Zdokumentovať požiadavky, ktoré sú zvyčajne definované priamo používateľom.
9. Umožniť vizualizáciu a pochopenie procesov pomocou prípadov použitia (*Use Case*).

Medzi konkrétne funkcionálne požiadavky navrhovanej aplikácie patrí:

- registrácia a prihlásenie (login) používateľov,
- verifikácia používateľa pri prihlasovaní,
- možnosť vytvorenia novej udalosti,

- pridávanie podrobností k udalosti (fotografie, popis, miesto konania, čas, typ udalosti, maximálny počet účastníkov a pod.),
- prehliadanie dostupných udalostí na interaktívnej mape,
- nastavenie polomeru (kruhu) pre zobrazovanie udalostí v okolí používateľa,
- filtrovanie udalostí podľa preferencií,
- správa účastníkov (možnosť odstrániť účastníka z vlastnej udalosti),
- ukladanie vybraných udalostí do zoznamu obľúbených,
- zobrazenie histórie (navštívené, odporúčané a nastávajúce udalosti),
- možnosť potvrdenia účasti na udalosti,
- komunikácia s ostatnými účastníkmi podujatia,
- hodnotenie udalostí,
- zdieľanie (*sharing*) podujatí,
- prehliadanie profilov ostatných používateľov,
- pridávanie priateľov,
- správa a nastavenie notifikácií,
- systém nahlasovania nevhodných používateľov alebo udalostí,
- možnosť zmeny jazyka v prostredí aplikácie,
- automatizovaný poradovník náhradníkov (*Waitlist*),
- systém penalizácie alebo zablokovania používateľa, ktorý potvrdil účasť, nedostavil sa a včas sa neodhlásil,
- odporúčací systém,
- možnosť hodnotenia udalostí,
- možnosť odísť z udalosti.

4.0.3 Nefunkcionálne požiadavky

Nefunkcionálne požiadavky definujú kvalitatívne vlastnosti systému a obmedzenia, za ktorých musí systém pracovať. Ich charakteristika je nasledovná:

1. Vysvetľujú vlastnosti produktu a jeho atribúty kvality.
2. Napomáhajú k pochopeniu výkonu a správania systému.
3. Identifikujú spôsob, akým má systém stanovené úlohy vykonávať.
4. Samotné nefunkcionálne požiadavky bez funkčných nepostačujú na chod systému.
5. Zabezpečujú, že systém bude spĺňať špecifické očakávania používateľov na kvalitu.
6. Sú žiaduce pre celkový zážitok, hoci v niektorých prípadoch nie sú kritické pre základnú funkcionalitu.
7. Ich splnenie zvyšuje konkurencieschopnosť a stabilitu softvéru.
8. Zvyčajne ich definujú softvéroví inžinieri, vývojári alebo architekti na základe technických potrieb.
9. Možno ich dokumentovať a chápať ako atribúty kvality (napr. bezpečnosť, rýchlosť).

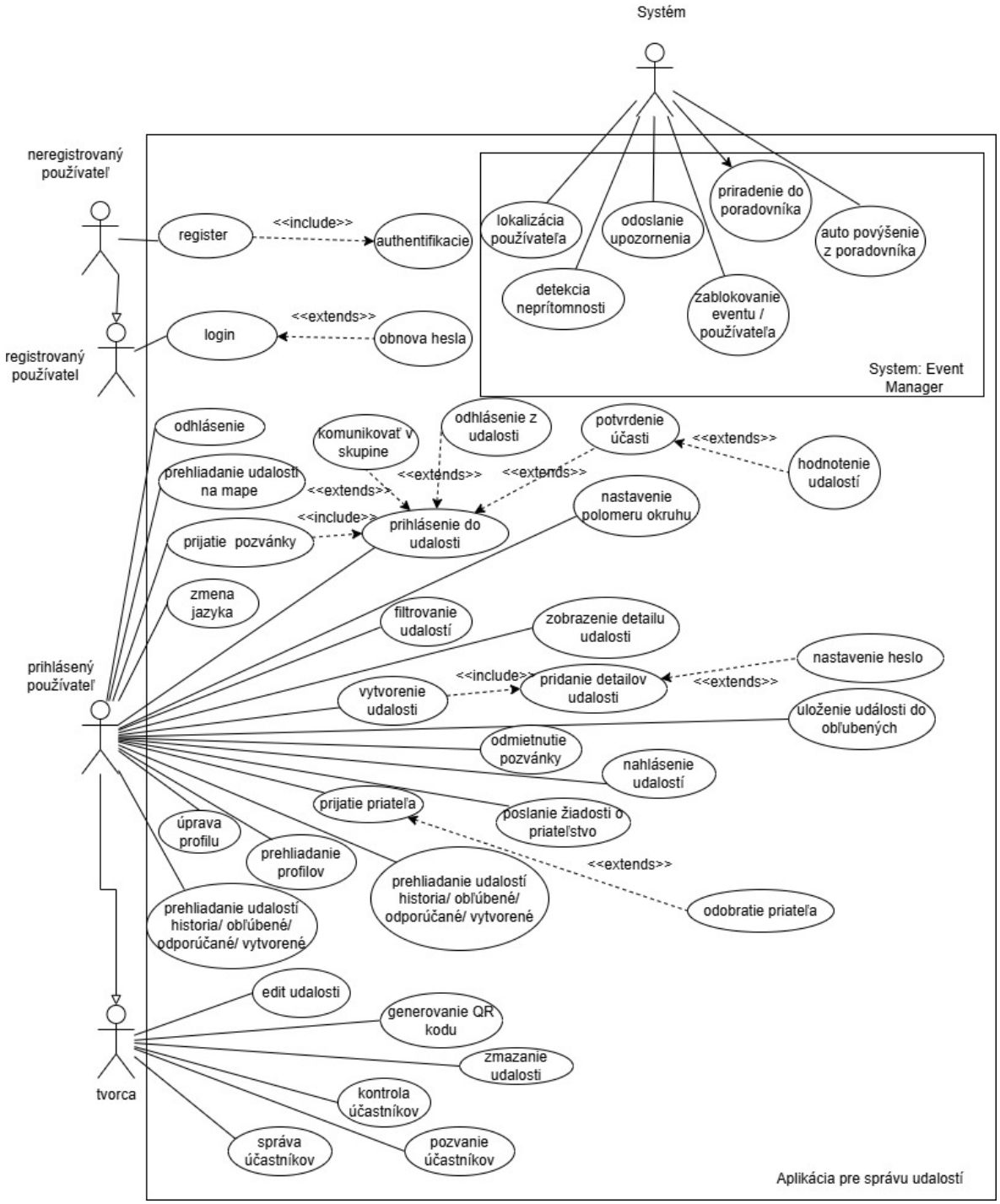
Pre navrhovanú aplikáciu boli definované tieto konkrétne nefunkcionálne požiadavky:

- **Intuitívnosť:** Používateľské rozhranie musí byť navrhnuté tak, aby bola navigácia v aplikácii jasná aj bez dodatočnej dokumentácie.
- **Responzivita:** Rozhranie aplikácie sa musí korektne prispôsobovať rôznym rozlíšeniam a pomerom strán Android zariadení.
- **Rýchlosť odozvy:** Systém musí zabezpečiť rýchlu odozvu pri načítavaní udalostí z mapy a pri spracovaní dát v odporúčacom algoritme (do 2 sekúnd).
- **Stabilita a funkčnosť:** Systém musí garantovať spoľahlivé prihlásenie používateľa a stabilitu procesov na pozadí.
- **Spracovanie a zaznamenávanie dát:** Efektívne ukladanie dát do databázy Firebase Firestore so zreteľom na minimalizáciu dátových prenosov.
- **Zaznamenávanie polohy:** Presné a šetrné spracovanie GPS súradníc používateľa pre potreby mapy a *Check-inu*.

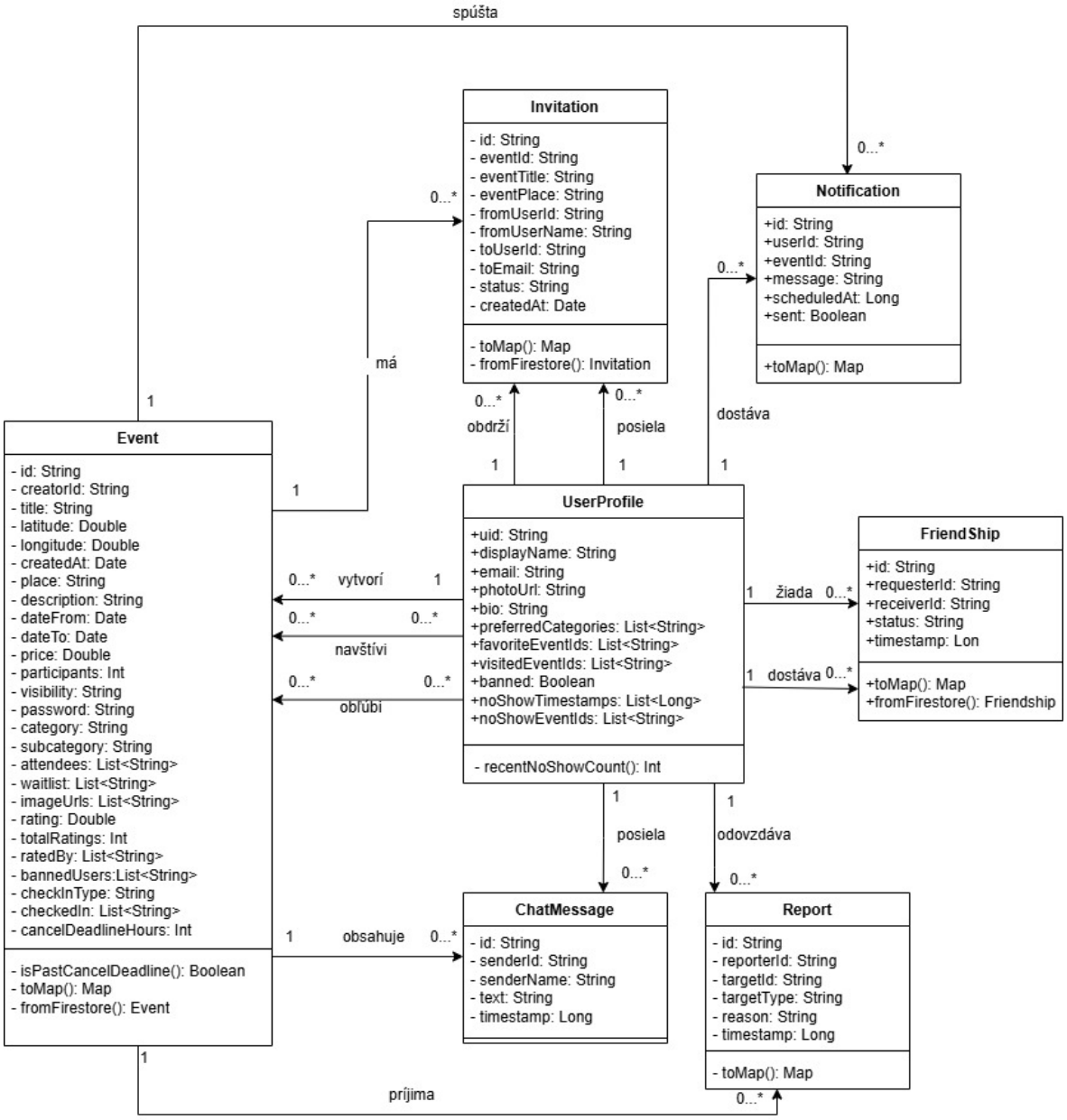
- **Správa povolení:** Systém musí korektne vyžadovať a spracovávať povolenia od používateľa (prístup k polohe, galérii a kalendáru).
- **Ochrana údajov:** Zabezpečenie osobných údajov používateľov a šifrovaná komunikácia medzi klientom a serverom.
- **Kompatibilita:** Možnosť inštalácie a synchrónneho používania na viacerých Android zariadeniach pod jedným účtom.
- **Rozšíriteľnosť:** Architektúra kódu musí umožňovať jednoduché pridávanie nových funkcií v budúcnosti bez nutnosti prepisovania jadra aplikácie.

4.0.4 UML diagramy

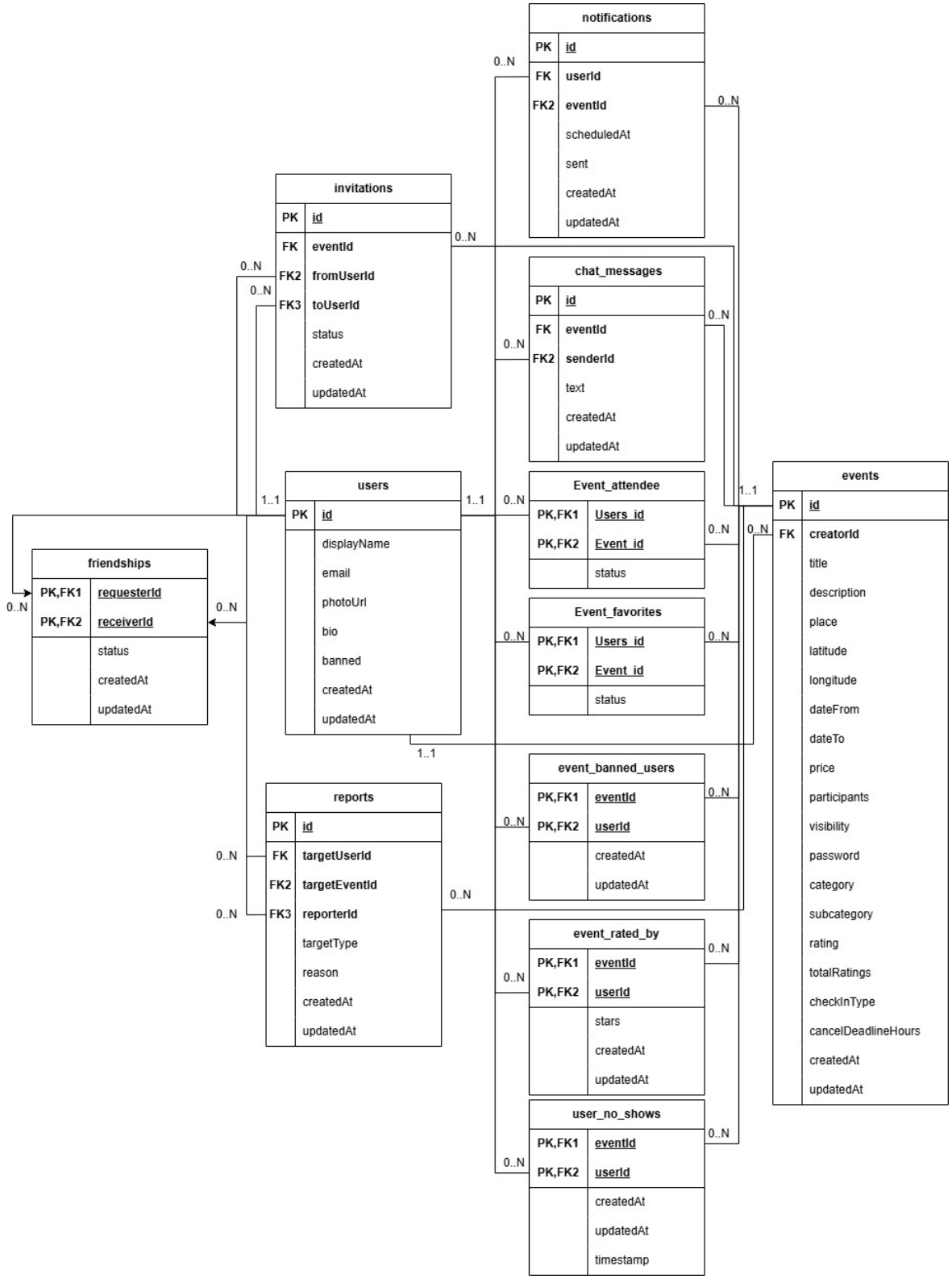
V tejto časti sú znázornené UML diagramy navrhovanej aplikácie, ktoré vizualizujú jej štruktúru a správanie.



Obr. 2: Diagram prípadov použitia (Use Case diagram)



Obr. 3: Diagram tried (Class diagram)



Obr. 4: Relačná databáza

Záver

„V tejto príručke sme sa snažili poskytnúť praktické pokyny pre vypracovanie záverečných prác na Fakulte elektrotechniky a informatiky STU v Bratislave. Obsahuje dôležité informácie o formálnej stránke dokumentu, ktoré sú nevyhnutné pre zabezpečenie jasnej štruktúry a správneho formátovania práce. Naša príručka kladie dôraz na dodržiavanie základných pravidiel, ktoré pomáhajú minimalizovať chyby a zlepšujú celkovú kvalitu predkladaných dokumentov.

Rovnakú pozornosť venujeme aspektom, ako sú formátovanie textu, zarovnávanie, riadkovanie a použitie preddefinovaných štýlov, ktoré prispievajú k profesionálnemu vzhľadu dokumentu. Dôkladné spracovanie matematických rovníc a grafov je ďalším dôležitým prvkom, ktorý môže ovplyvniť hodnotenie práce. Práca s týmito aspektmi zaisťuje, že obsah je prehľadný a presne odráža zamýšľané myšlienky.

V neposlednom rade sa naša príručka zameriava na význam citovania externých zdrojov, čo je zásadné pre zachovanie integrity a etických štandardov pri písaní akademických textov. Správne citovanie nielenže posilňuje argumentáciu, ale tiež pomáha predchádzať plagiátorstvu, ktoré môže mať vážne následky na akademickú kariéru.

Dodržiavanie týchto pokynov umožňuje vyhnúť sa bežným chybám a zvyšuje kvalitu záverečných prác, čím sa zlepšuje celkový dojem z akademických schopností. Dobre vypracovaná práca môže viesť k pozitívnemu hodnoteniu a uznaniu zo strany hodnotiteľov, čo je cieľom každého študenta.

Celkovo je príručka navrhnutá ako cenný nástroj na podporu pri písaní záverečných prác. Poskytnuté informácie a odporúčania prispejú k dosiahnutiu akademických cieľov a pomôžu vypracovať kvalitné a odborné dokumenty, ktoré budú spĺňať štandardy fakulty.“

(ChatGPT 3, 2024)

Poznámka autora: Časť záveru vygenerovala umelá inteligencia na základe analýzy obsahu verzie dokumentu pre práce písané v MS Worde. Konkrétne išlo o aplikáciu ChatGPT 3 dňa 29. 9. 2024. Rukopis umelej inteligencie je z textu cítiť aj napriek tomu, že celá komunikácia vedúca k tejto forme záveru trvala približne 40 minút. Parafrázujúc ľudové príslovie konštatujeme, že umelá inteligencia je možno dobrý sluha, ale zlý pán. Používajme ju s rozumom.

Pripomíname, že takýto spôsob tvorby obsahu je v záverečnej práci, ako aj v iných akademických textoch, neprípustný. Autorské dielo nesmie obsahovať odseky vytvorené generatívnou umelou inteligenciou. Preto sme ho formátovali ako citovaný text.

Použitie nástrojov umelej inteligencie

OpenAI (2025), ChatGPT 4o, časť **??**, generovanie vzorca záznamu použitia AI.

OpenAI (2024), ChatGPT 3, Záver, generovanie textu.

OpenAI (2025), ChatGPT 4o, dodatok B, generovanie kódu programu.

OpenAI (2024), ChatGPT 3, dodatok C, generovanie zoznamu.

Dodatok A: Algoritmus

Algoritmus 1 Vypočítaj $y = x^n$

Require: $n \geq 0 \vee x \neq 0$

Ensure: $y = x^n$

```
y ⇐ 1
if n < 0 then
    X ⇐ 1/x
    N ⇐ −n
else
    X ⇐ x
    N ⇐ n
end if
while N ≠ 0 do
    if N is even then
        X ⇐ X × X
        N ⇐ N/2
    else {N is odd}
        y ⇐ y × X
        N ⇐ N − 1
    end if
end while
```

Dodatok B: Výpis dlhého kódu

Dlhšie kódy, prípadne kompletné programy vypisujeme do dodatkov práce. Nasledujúci program v jazyku Python vygeneruje súbor `thesis.tex`, ktorý je funkčne totožný s hlavným súborom tohto dokumentu. Stačí vyplniť prehľadný formulár pri volaní funkcie `generate_thesis_tex` na konci kódu a získate hlavný súbor vašej práce.

```
def generate_thesis_tex(
    author,
    reg_nr,
    title,
    title_en,
    day,
    month,
    year,
    keywords,
    keywords_en,
    study_programme,
    study_programme_en,
    study_field,
    study_field_en,
    training_workplace,
    training_workplace_en,
    supervisor,
    consultant=None,
    thesis_type="bp",
    language="sk",
):

    tex_class_option = thesis_type if thesis_type in ["bp", "dp"] else "bp"
    tex_language_option = "en" if language == "en" else ""

    if tex_language_option:
        document_class = f"\\documentclass[{tex_class_option},{tex_language_option
    }]{{FEIstyle}}"
    else:
        document_class = f"\\documentclass[{tex_class_option}]{{FEIstyle}}"

    tex_content = f"""
{document_class}

% Author of the thesis
\\FEIauthor{{{author}}}
```

```
% Evidence number
\\FEIregNr{{{reg_nr}}}

% Title
\\FEItitle{{{title}}}
\\FEItitleEn{{{title_en}}}

% Date
\\FEIdate{{{day}}}{{{month}}}{{{year}}}

% Keywords
\\FEIkeywords{{{keywords}}}
\\FEIkeywordsEn{{{keywords_en}}}

% Further details
\\FEIstudyProgramme{{{study_programme}}}
\\FEIstudyProgrammeEn{{{study_programme_en}}}
\\FEIstudyField{{{study_field}}}
\\FEIstudyFieldEn{{{study_field_en}}}
\\FEItrainingWorkplace{{{training_workplace}}}
\\FEItrainingWorkplaceEn{{{training_workplace_en}}}

% Supervisor
\\FEIsupervisor{{{supervisor}}}
"""

    if consultant:
        tex_content += f"\n\\FEIconsultant{{{consultant}}}\n"

    tex_content += f"""

% Bibliography database file
\\bibliography{{includes/bibliography.bib}}

\\begin{{document}}

\\frontmatter
\\FEIpdfInfo
\\FEIcover
\\FEItitlePage
\\includepdf[pages=-]{{includes/assignment.pdf}}
\\FEIthanks{{includes/thanks}}
\\FEIabstract{{includes/abstract}}
\\FEIabstractEn{{includes/abstractEN}}
```

```
\\FEIcontent
\\FEImanualListOfGlossaries{{includes/manual_glossary}}
\\FEIlistOfAlgorithms
\\FEIlistOfListings

\\mainmatter
\\FEIintroduction{{includes/introduction}}
\\FEIcore{{includes/core}}
\\FEIconclusion{{includes/conclusion}}
\\FEIbibliography
\\FEIaiDeclaration{{includes/ai_declaration}}
\\backmatter
\\FEIappendix{{Algoritmus\\label{{att:algorithms}}}}{{includes/attachmentA}}
\\FEIappendix{{Výpis dlhého kódu\\label{{att:listings}}}}{{includes/attachmentB}}
\\FEIappendix{{Slovníček pojmov\\label{{att:dictionary}}}}{{includes/attachmentC}}

\\end{{document}}
"""

    with open("thesis.tex", "w", encoding="utf-8") as file:
        file.write(tex_content)

    print("Súbor thesis.tex bol úspešne vygenerovaný.")


# Testovacie volanie funkcie
generate_thesis_tex(
    author="RNDr. Juraj Chlpík, PhD.",
    reg_nr="FEI-xxxx-xxxx",
    title="Rozšírená šablóna záverečnej práce na FEI STU v Bratislave v systéme
    LaTeX",
    title_en="Extended thesis template at FEI STU in Bratislava in LaTeX system",
    day="31",
    month="12",
    year="2024",
    keywords="záverečná práca, šablóna, LaTeX, formátovanie textu, citácie",
    keywords_en="Final thesis, template, LaTeX, text formatting, citations",
    study_programme="názov študijného programu",
    study_programme_en="Study program in English",
    study_field="názov študijného odboru",
    study_field_en="Field of the study in English",
    training_workplace="Názov školiaceho pracoviska",
    training_workplace_en="Training Work place",
    supervisor="tituly Meno Priezvisko, tituly",
    consultant="",
```

```
    thesis_type="bp",  # Možnosti: "bp", "dp"
    language="sk",  # Možnosti: "sk" (slovenčina - predvolená), "en" (angličtina)
)
```

Výpis kódu 1: Ukážka výpisu kódu programu načítaného z externého súboru pomocou makra `\lstinputlisting`

Dodatok C: Slovníček pojmov

Abstrakt – Krátke zhrnutie hlavných bodov dokumentu, vrátane cieľov, metodológie a výsledkov.

Bod (pt) – Jednotka merania veľkosti písma; 1 bod sa rovná približne 0,352 mm.

Bold – Písmo, ktoré je tučné, používané na zvýraznenie dôležitých častí textu a v nadpisoch.

Citácia – Uvedenie zdroja informácií v texte, ktoré umožňuje identifikáciu pôvodného materiálu.

Font – Typ písma, ktorý sa používa na zobrazenie textu. Rôzne fonty majú odlišný štýl a vzhľad.

Graf – Vizualizácia údajov, ktorá zobrazuje vzťah medzi premennými.

Kapitola – Hlavná časť dokumentu, ktorá sa zaoberá konkrétnou témou a je zvyčajne označená číslom alebo názvom.

Korektúra – Proces revízie a opravy textu s cieľom odstrániť chyby.

Kurzíva – Typ písma, ktorý je sklonený doprava a používa sa na zvýraznenie textu, názvov alebo cudzích výrazov.

Metodológia – Opis metód a techník použitých pri výskume alebo analýze.

Normostrana – Základná jednotka pre meranie dĺžky textu, ktorá zvyčajne obsahuje 1 800 znakov vrátane medzier.

Obrázok – Grafický prvok alebo fotografia, ktorá ilustruje text.

Odsadenie – Vzdialenosť prvého riadku odseku od okraja stránky.

Odsek – Základná jednotka textu, ktorá obsahuje súvislé myšlienky. V LaTeX-u sa odseky oddeľujú prázdnym riadkom v zdrojovom texte.

Plagiátorstvo – Prezentovanie cudzej práce alebo myšlienok ako vlastných bez riadneho citovania.

Rez písma – Variant typu písma, ktorý určuje štýlové úpravy, ako sú normálny, tučný (bold), kurzíva (italic) alebo podčiarknutie.

Riadkovanie – Vzdialenosť medzi jednotlivými riadkami textu.

Sadzba – Proces usporiadania textu a grafiky na stránke.

Súbor – Elektronický dokument, ktorý obsahuje text alebo dáta.

Šablóna – Prednastavené formátovanie pre rôzne typy textu, ktoré zabezpečuje jednotný vzhľad.

Tabuľka – Usporiadanie údajov do riadkov a stĺpcov.

Titulný list – Prvá stránka dokumentu, ktorá obsahuje základné informácie, ako je názov práce a meno autora.

Typografia – Umenie a technika usporiadania textu tak, aby bol esteticky príťažlivý a čitateľný.

Úvod – Prvá časť dokumentu, ktorá predstavuje tému a ciele výskumu.

Vedecká práca – Akademický dokument, ktorý prezentuje výsledky výskumu, analýzy alebo diskusie o špecifickej téme.

Zdroje primárne a sekundárne – Primárne zdroje sú originálne dokumenty; sekundárne zdroje interpretujú alebo analyzujú primárne zdroje.